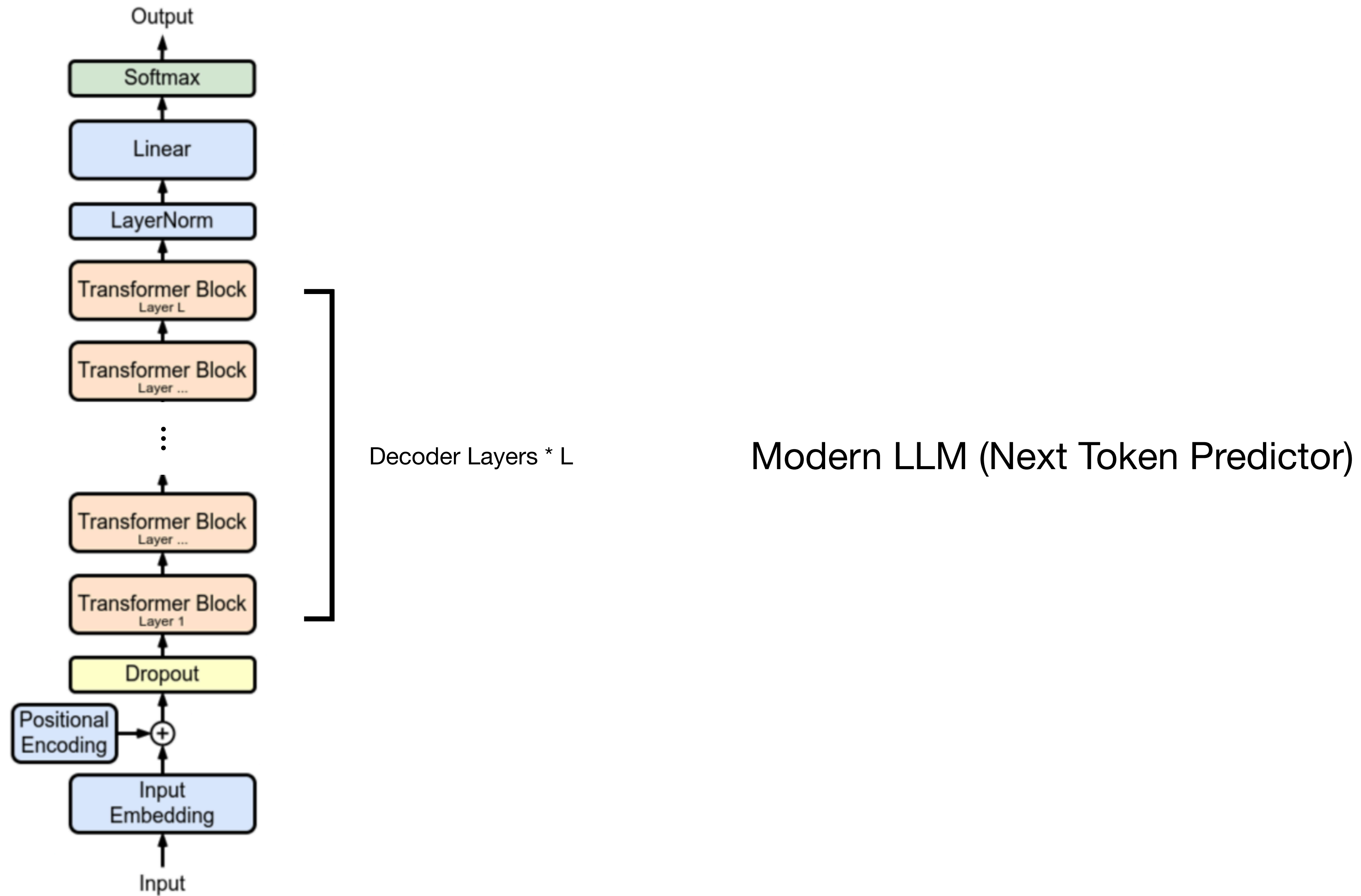
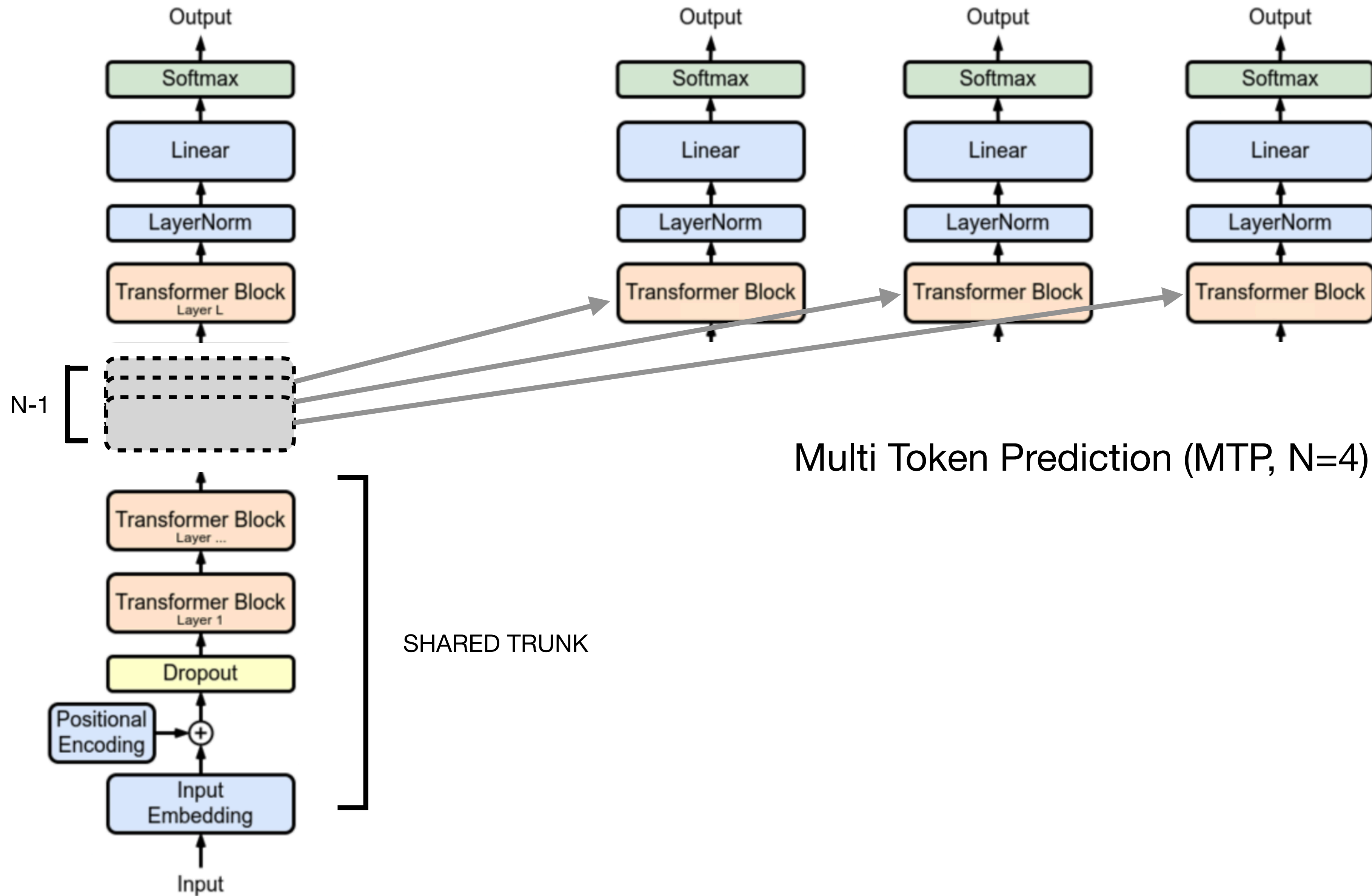
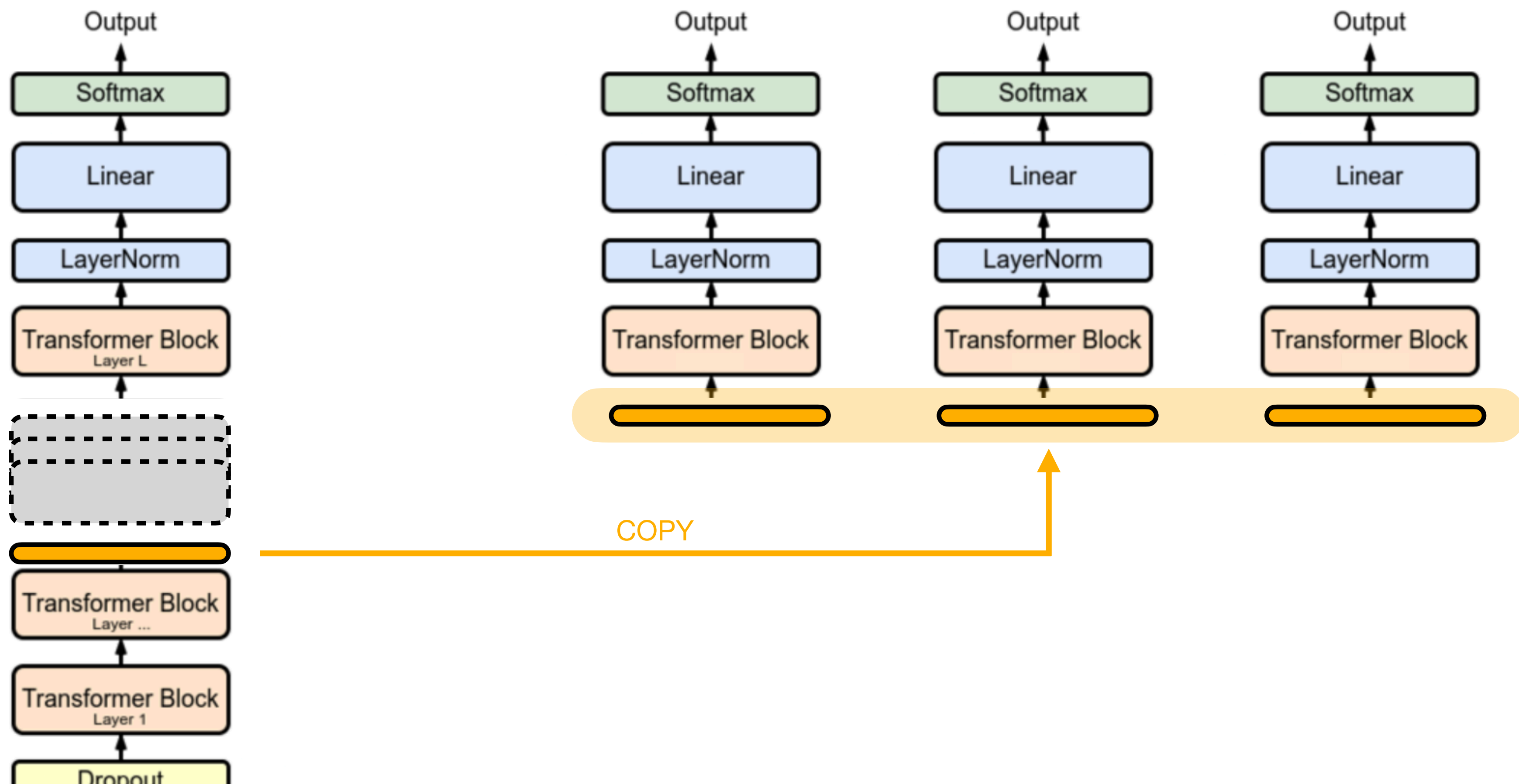


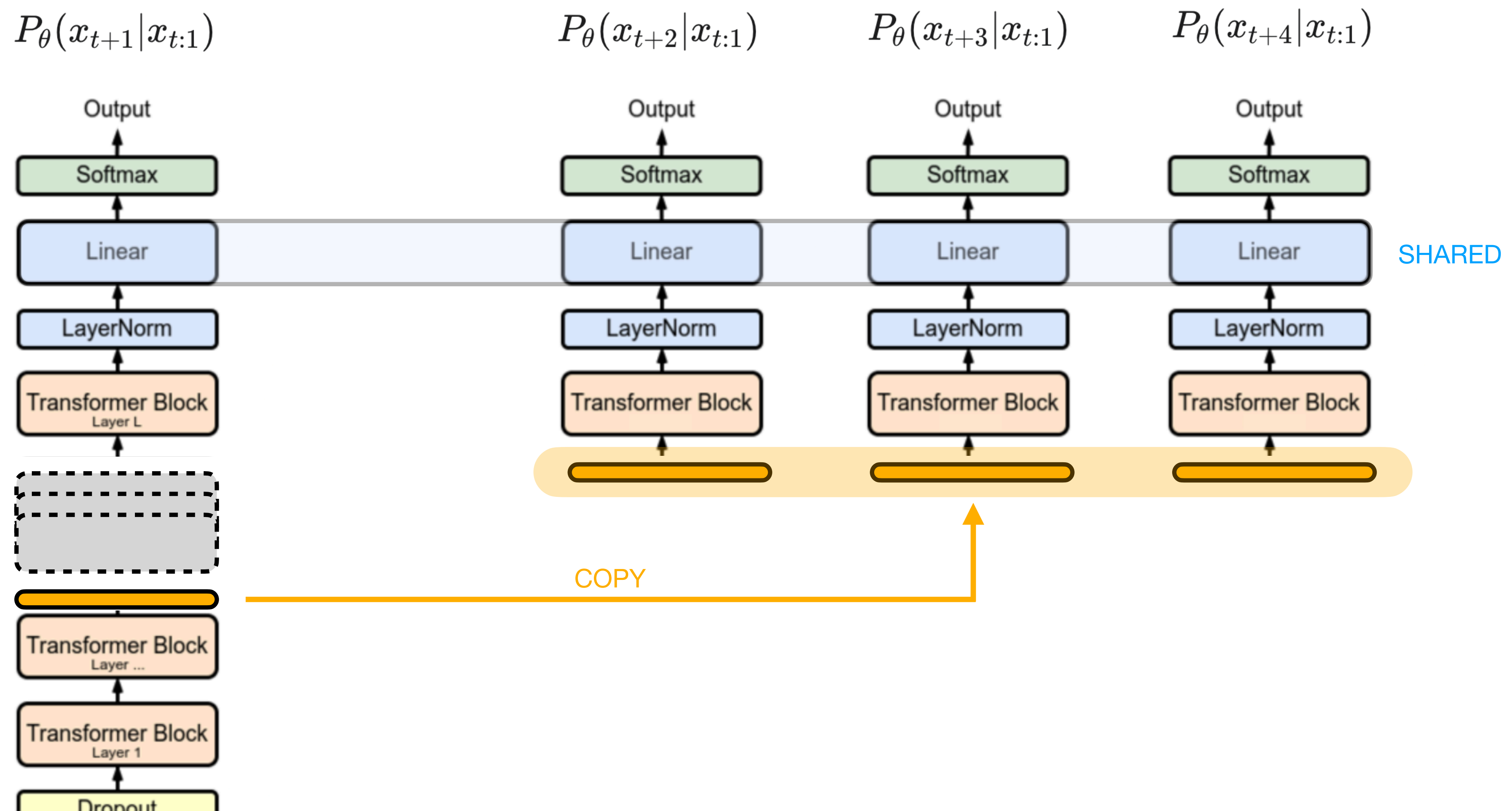
Value model을 활용한 MTP모델의 Weighted-SFT

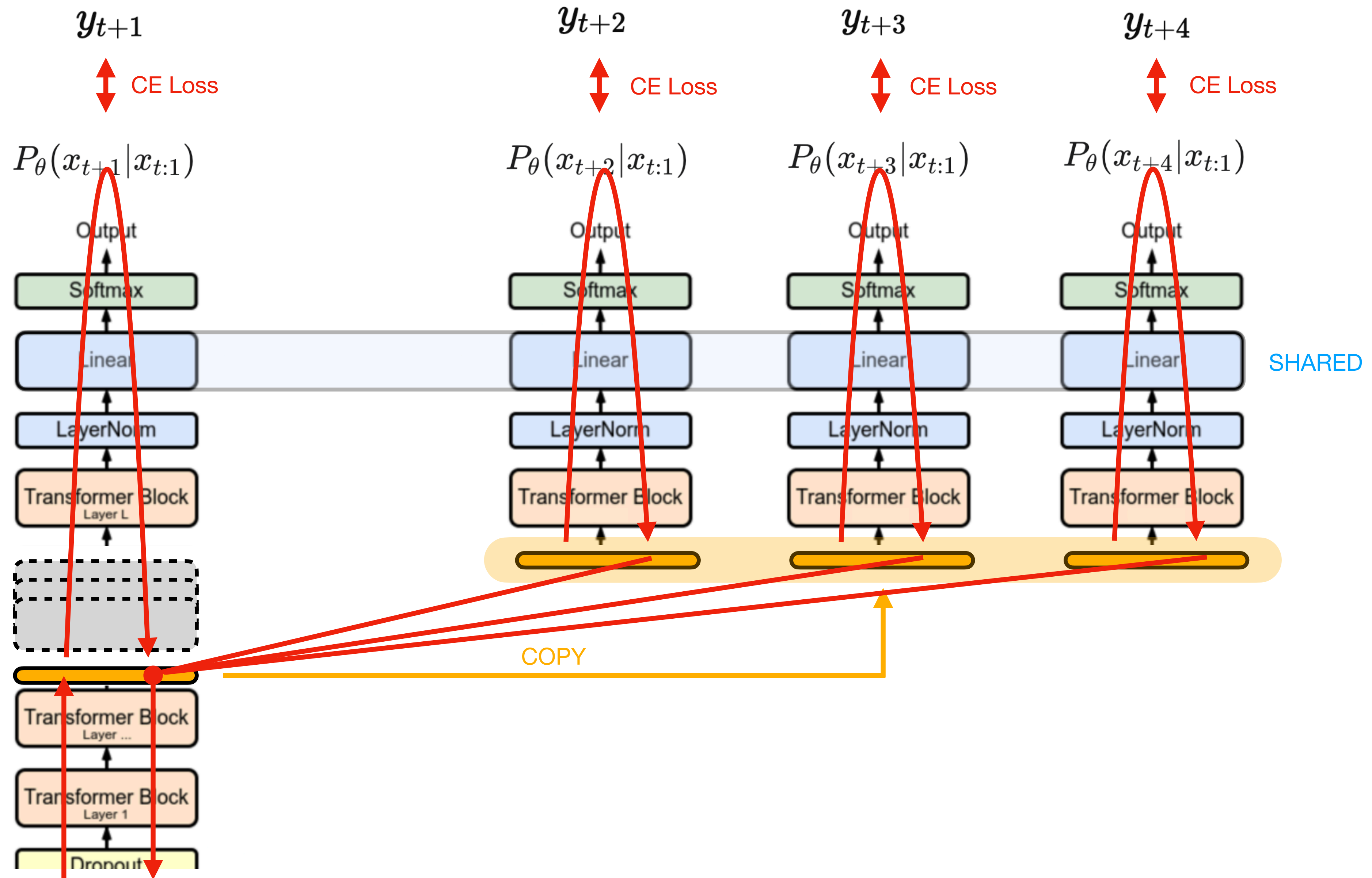
DeepSix (김상호, 김은경, 원우식, 정이우, 한영재, 홍혜진)











$$L_1 = - \sum_t \log P_\theta(x_{t+1} \mid x_{t:1}),$$

Humanity has condensed its most ingenious undertakings, surprising **findings**

Humanity has condensed its most ingenious undertakings, surprising findings **and**

Humanity has condensed its most ingenious undertakings, surprising findings and **beautiful**

Humanity has condensed its most ingenious undertakings, surprising findings and beautiful **productions**

$$L_n = - \sum_t \log P_\theta(x_{t+n:t+1} \mid x_{t:1}).$$

Humanity has condensed its most ingenious undertakings, surprising **findings** and beautiful productions

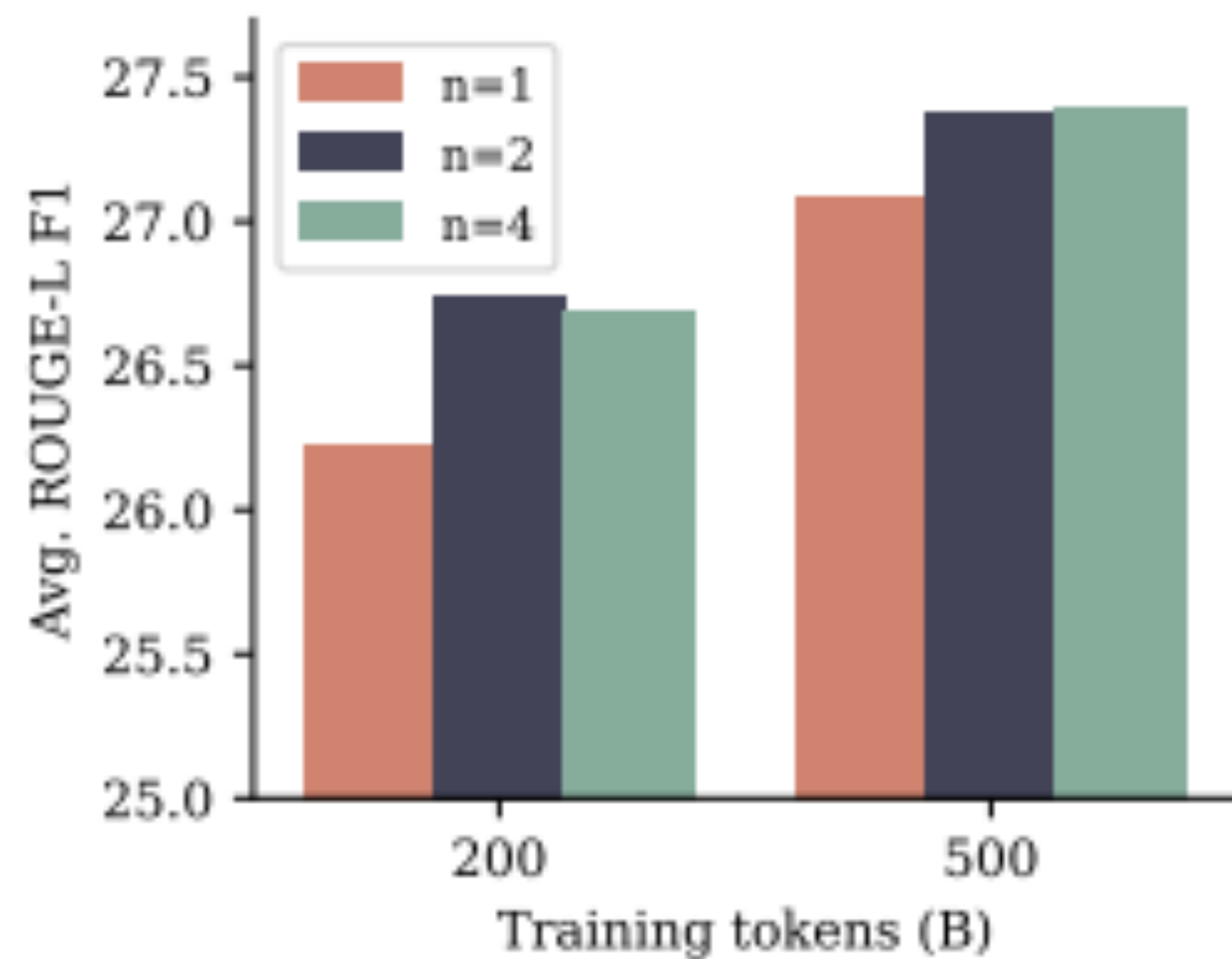
Humanity has condensed its most ingenious undertakings, surprising findings **and** beautiful productions into

Humanity has condensed its most ingenious undertakings, surprising findings and **beautiful** productions into text

Humanity has condensed its most ingenious undertakings, surprising findings and beautiful **productions** into text .

Why MTP?

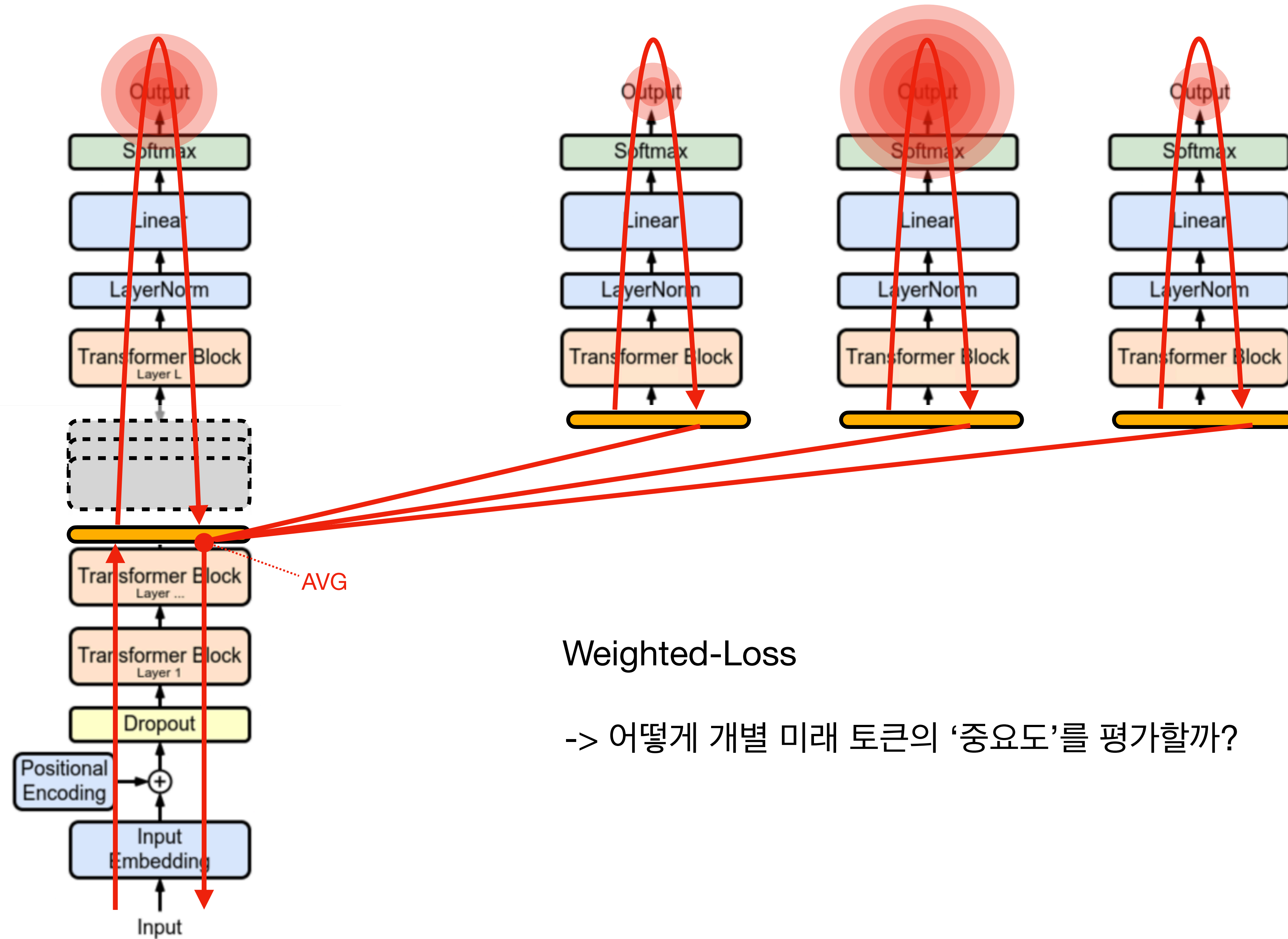
Humanity has condensed its most ingenious undertakings, surprising findings and beautiful productions into text. Large Language Models (LLMs) trained on all of these corpora are able to extract impressive amounts of world knowledge, as well as basic reasoning capabilities by implementing a simple—yet powerful—unsupervised learning task: next-token prediction. Despite the recent wave of impressive achievements ([OpenAI, 2023](#)), next-token prediction remains an inefficient way of acquiring language, world knowledge and reasoning capabilities. More precisely, teacher forcing with next-token prediction latches on local patterns and overlooks “hard” decisions. Consequently, it remains a fact that state-of-the-art next-token predictors call for orders of magnitude more data than human children to arrive at the same level of fluency ([Frank, 2023](#)).



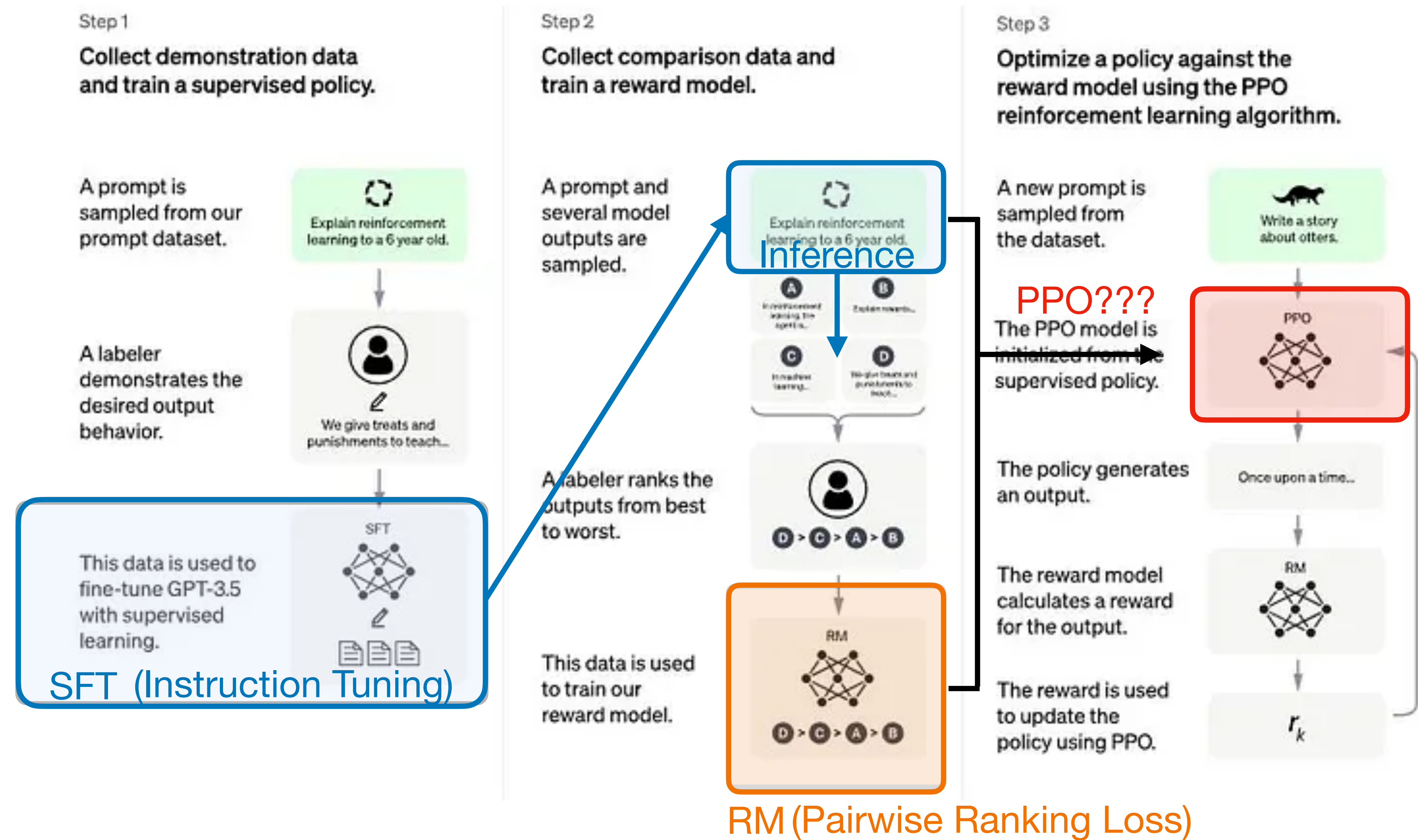
Training data	Vocabulary	n	MBPP			HumanEval			APPS/Intro		
			@1	@10	@100	@1	@10	@100	@1	@10	@100
313B bytes (0.5 epochs)	bytes	1	19.3	42.4	64.7	18.1	28.2	47.8	0.1	0.5	2.4
		8	32.3	50.0	69.6	21.8	34.1	57.9	1.2	5.7	14.0
		16	28.6	47.1	68.0	20.4	32.7	54.3	1.0	5.0	12.9
		32	23.0	40.7	60.3	17.2	30.2	49.7	0.6	2.8	8.8
200B tokens (0.8 epochs)	32k tokens	1	30.0	53.8	73.7	22.8	36.4	62.0	2.8	7.8	17.4
		2	30.3	55.1	76.2	22.2	38.5	62.6	2.1	9.0	21.7
		4	33.8	55.9	76.9	24.0	40.1	66.1	1.6	7.1	19.9
		6	31.9	53.9	73.1	20.6	38.4	63.9	3.5	10.8	22.7
		8	30.7	52.2	73.4	20.0	36.6	59.6	3.5	10.4	22.1
1T tokens (4 epochs)	32k tokens	1	40.7	65.4	83.4	31.7	57.6	83.0	5.4	17.8	34.1
		4	43.1	65.9	83.7	31.6	57.3	86.2	4.3	15.6	33.7

우리가 던진 질문은 ?

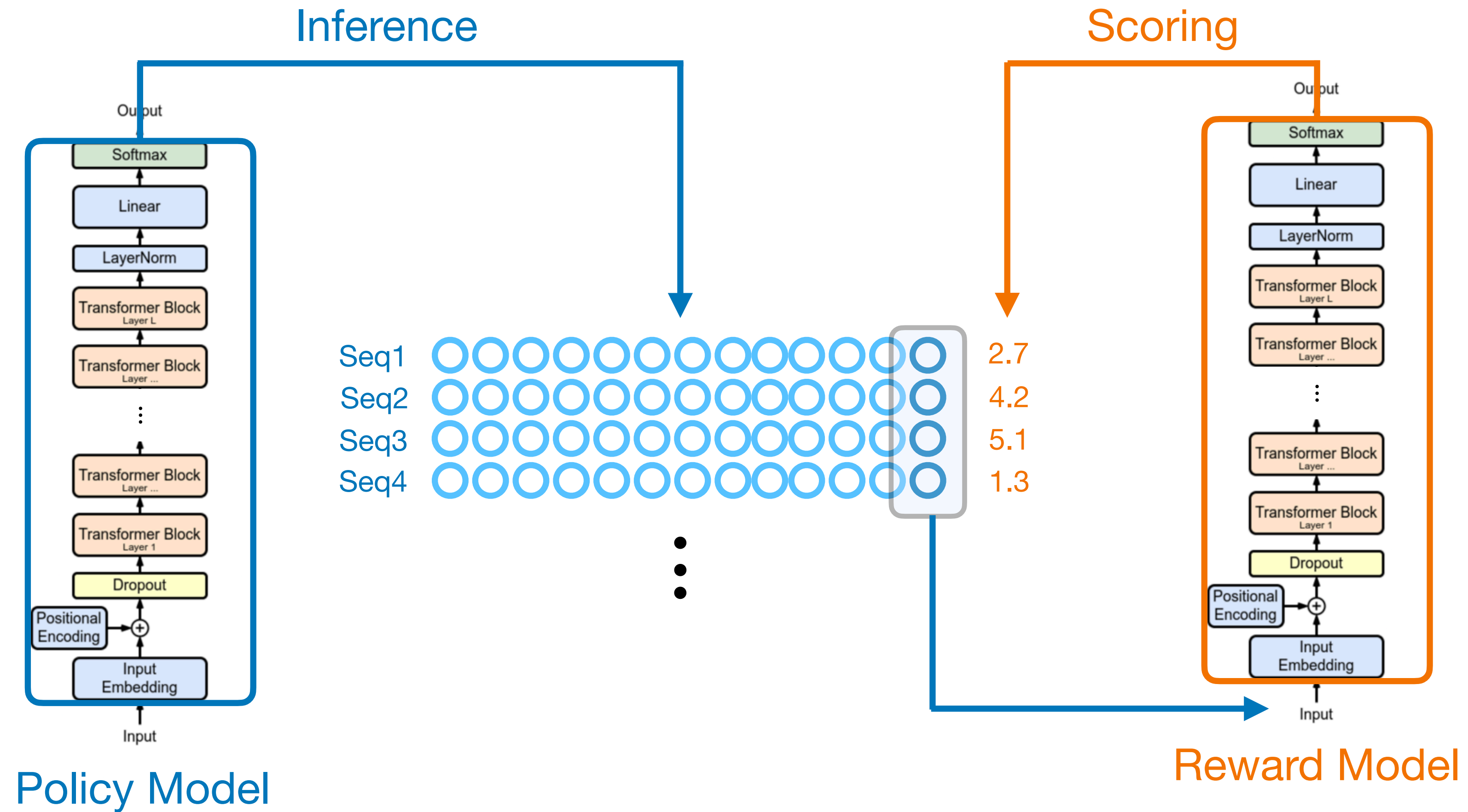
1. N개의 미래 token은 모두 ‘동등하게’ 중요한가?
2. 중요한 미래 token만 선별적으로 학습할 수는 없을까?



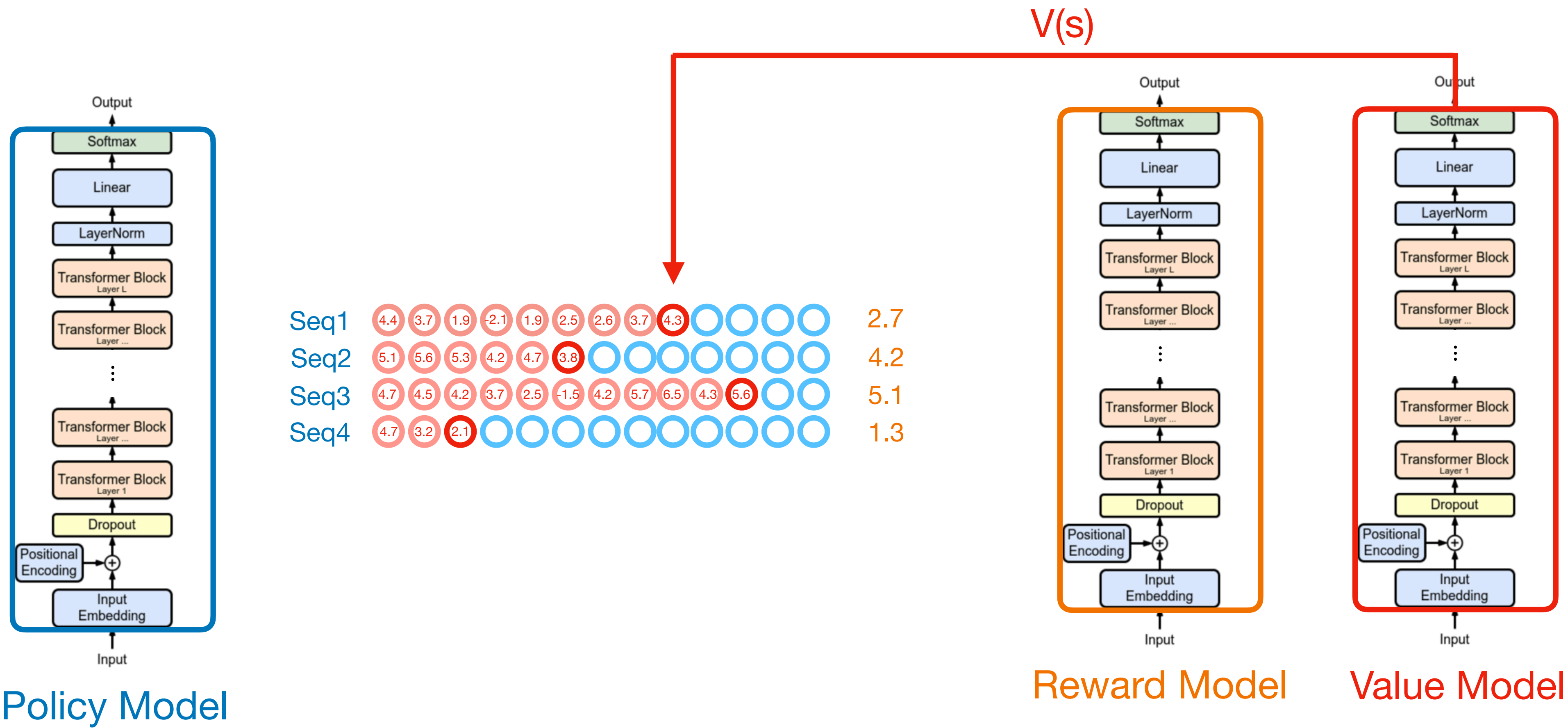
Reinforcement Learning



PPO (Proximal Policy Optimization) RLHF



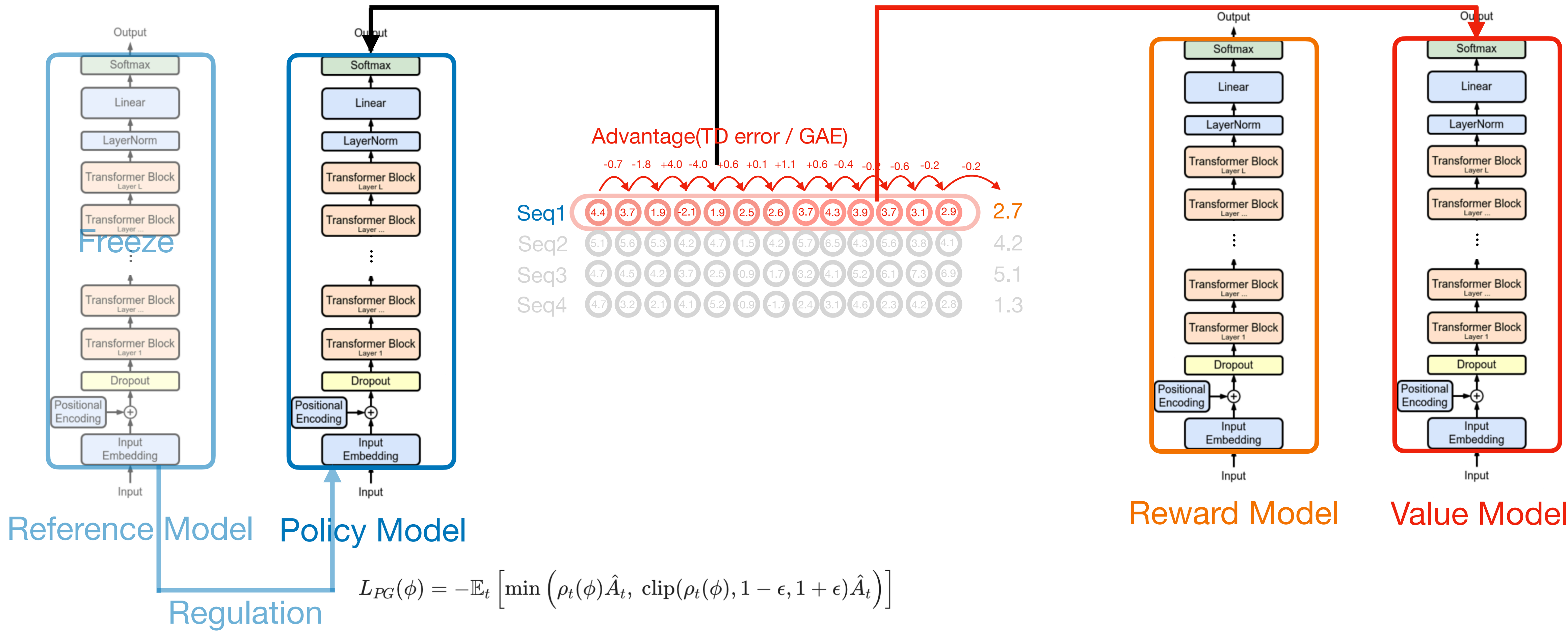
PPO (Proximal Policy Optimization) RLHF



PPO (Proximal Policy Optimization) RLHF

$$L_{PG}(\theta) = -\frac{1}{T} \sum_{t=0}^T \log \pi_{\theta}(a_t | s_t) \cdot \hat{A}_t$$

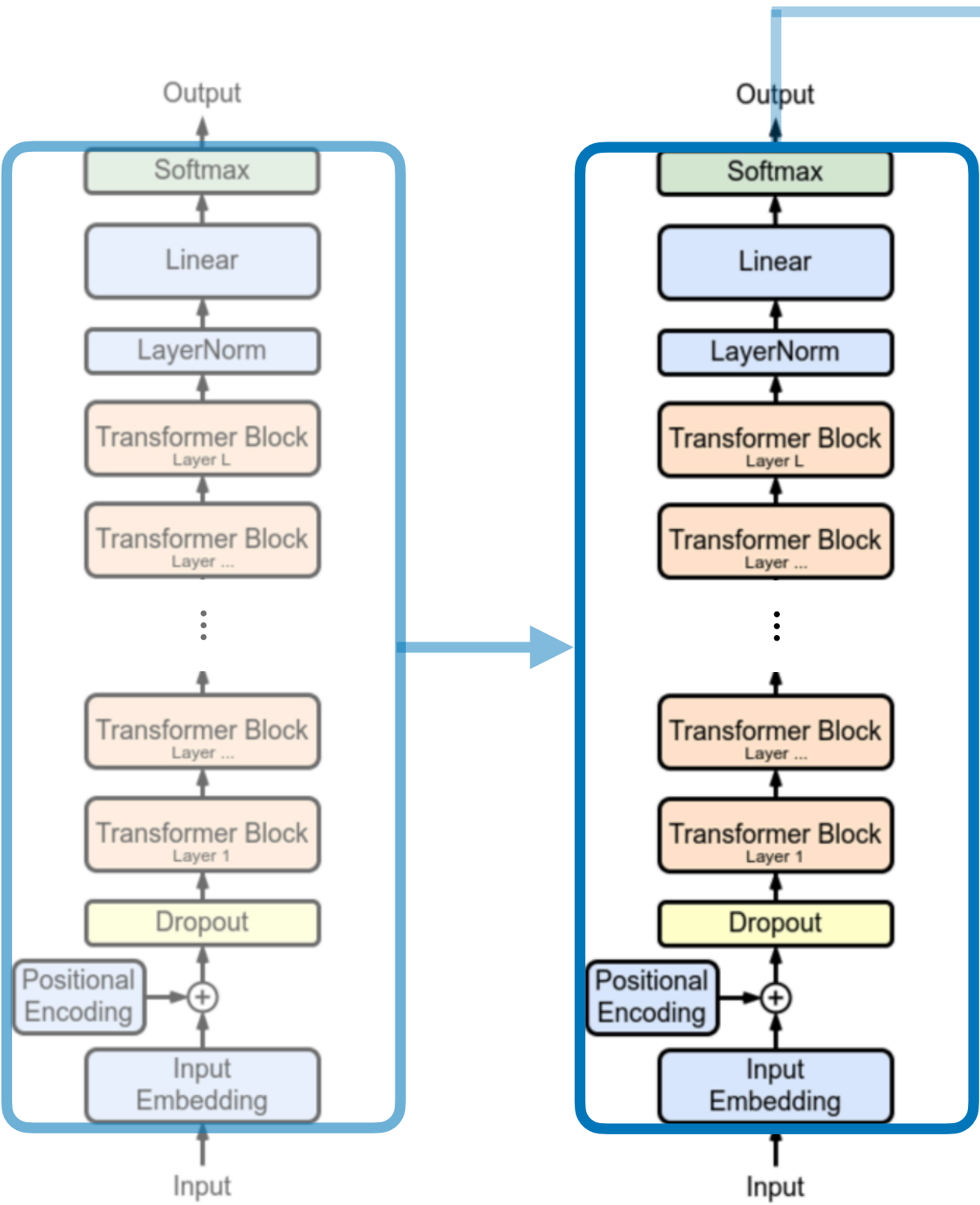
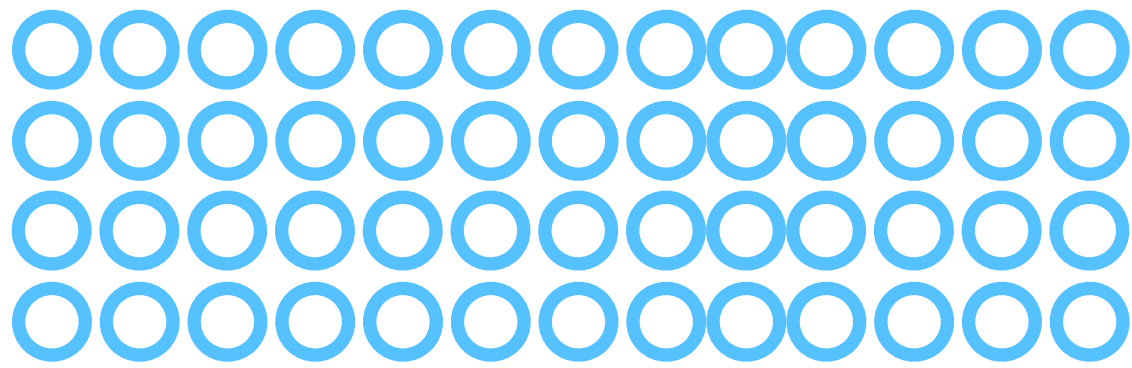
$$L_{VF}(\phi) = \frac{1}{T} \sum_{t=0}^T (V_{\phi}(s_t) - V_t^{\text{target}})^2$$



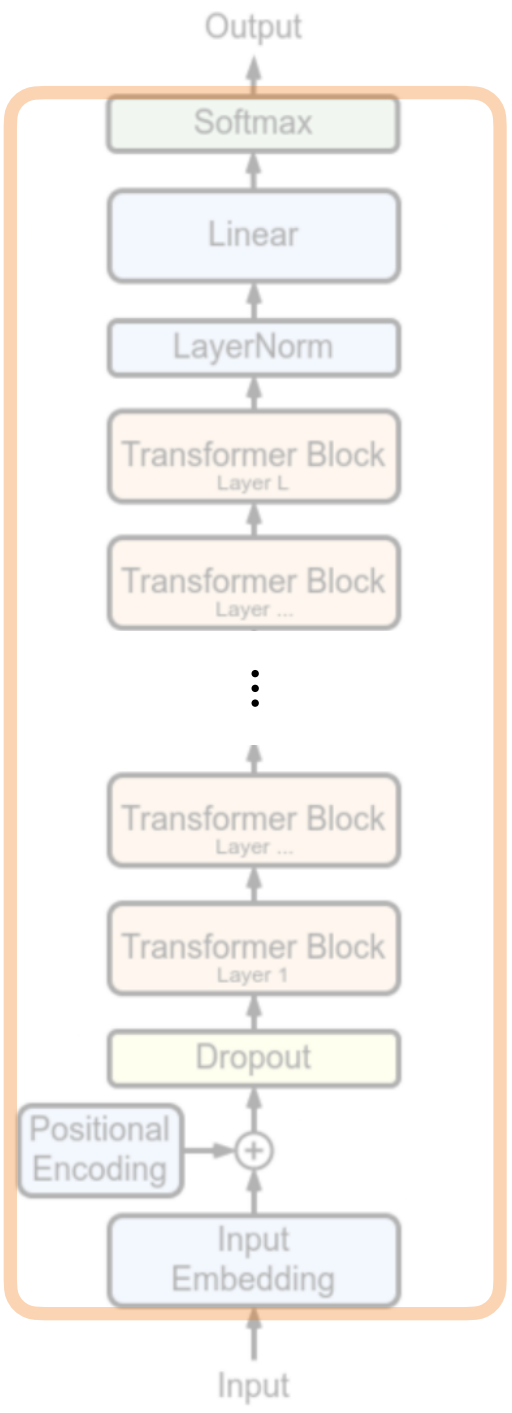
SFT와 RL의 차이점 : Online Data가 필요하지 않다

~~Inference~~

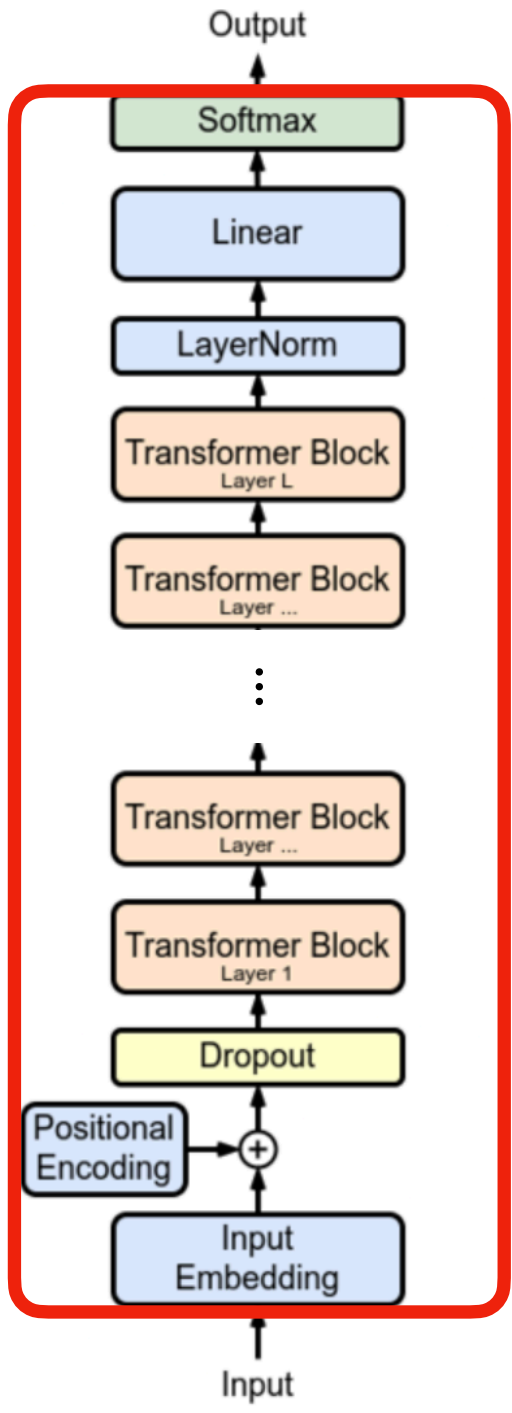
Offline Data



Reference Model Policy Model



Reward Model

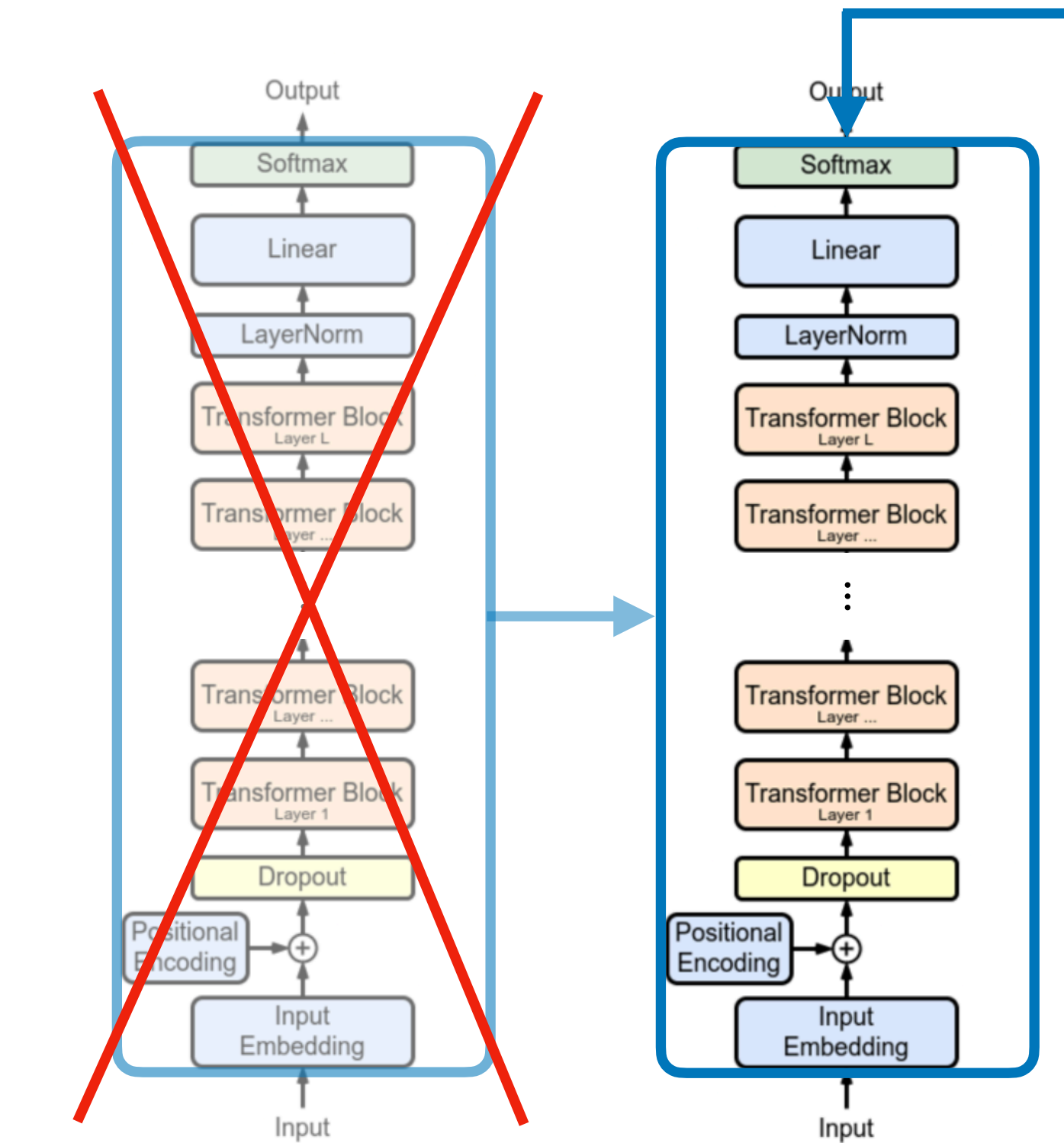
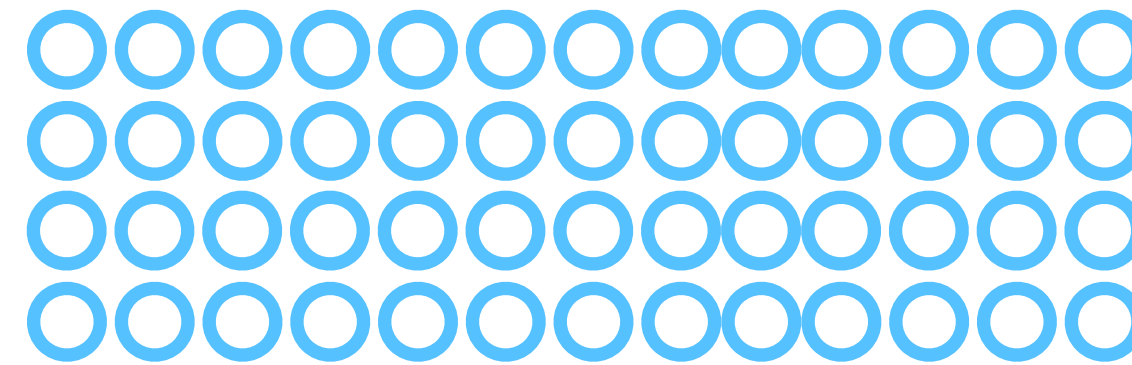


Value Model

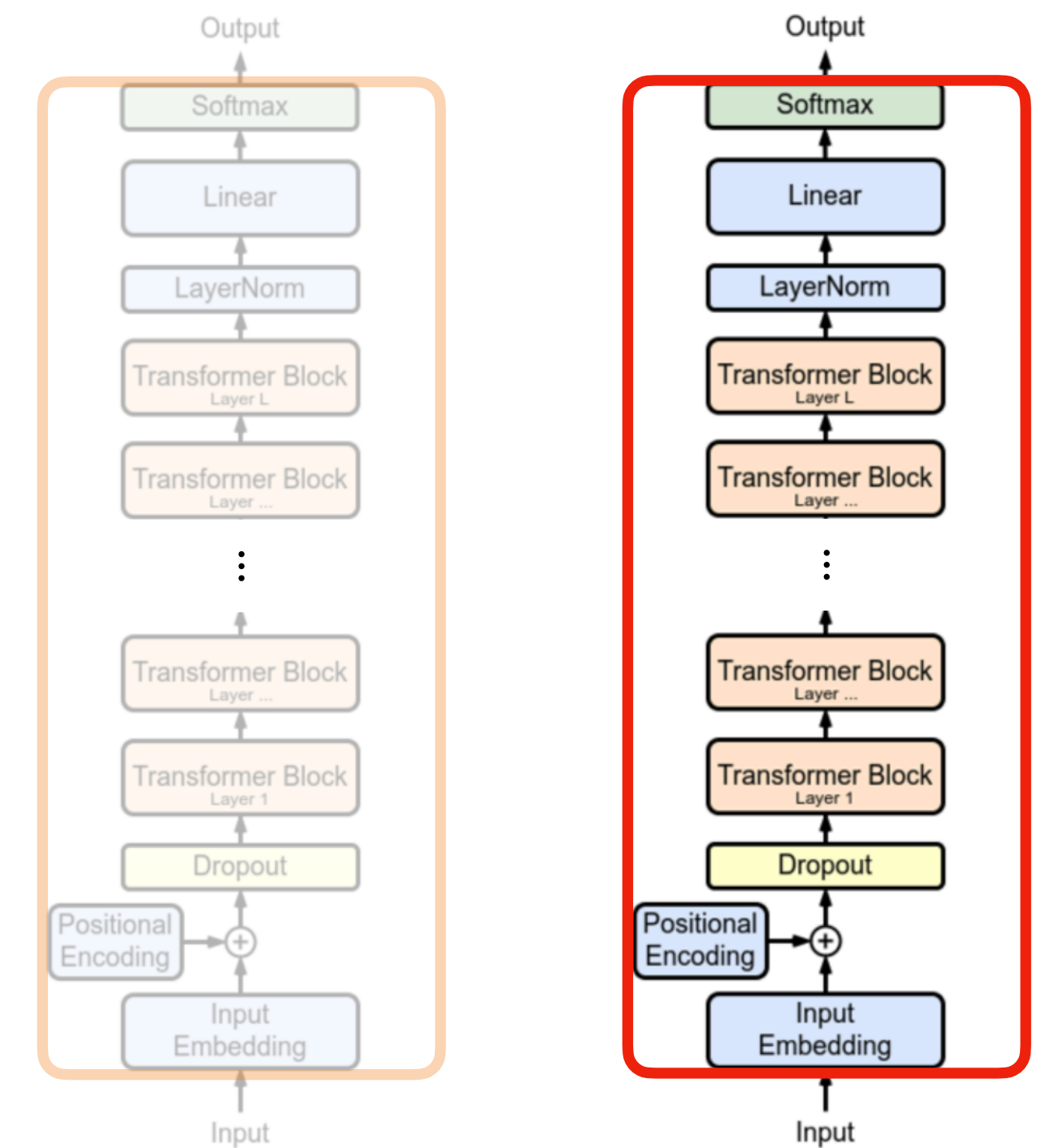
SFT와 RL의 차이점 : Ref Model이 필요하지 않다

SFT

Offline Data



Reference Model Policy Model



Reward Model

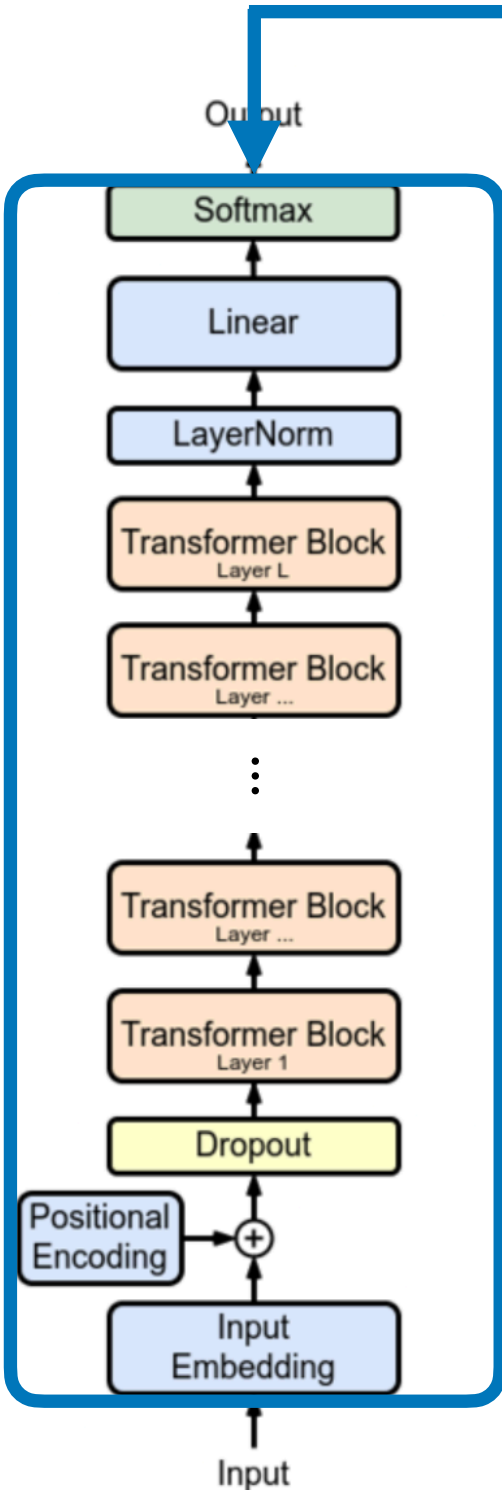
Value Model

Verifiable DATA : RM이 필요하지 않다

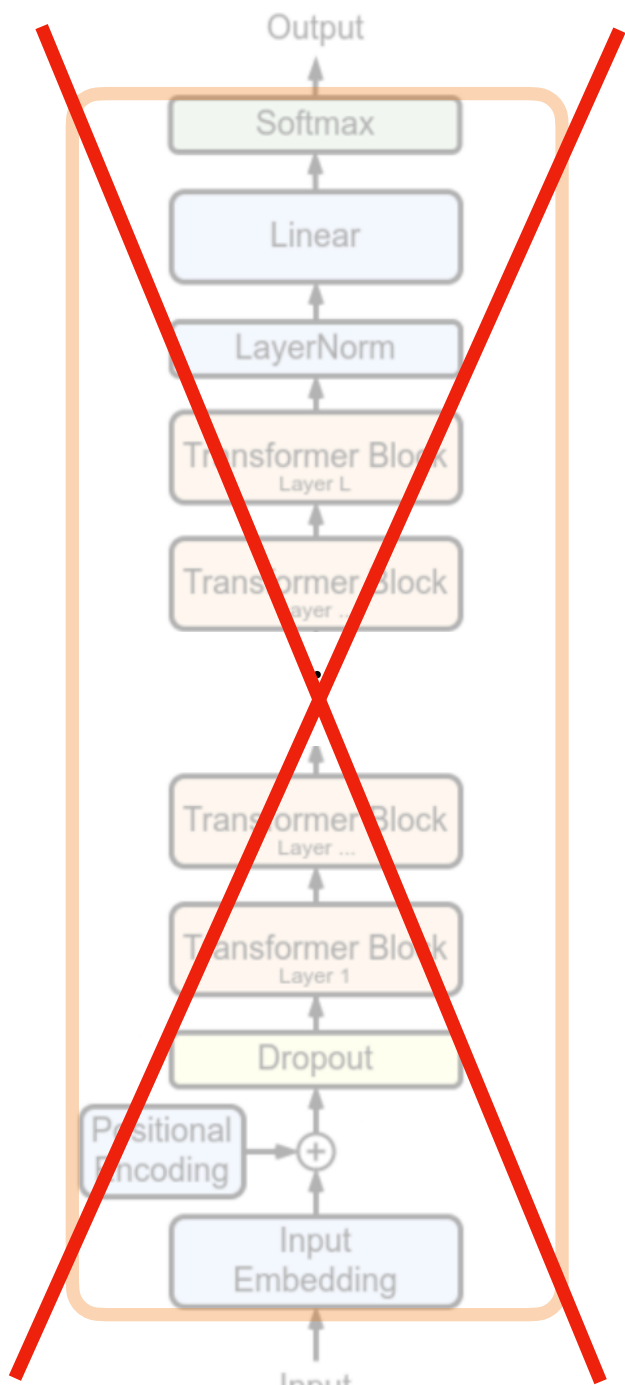
SFT

Offline “Verifiable” Data
(e.g. Codecontests)

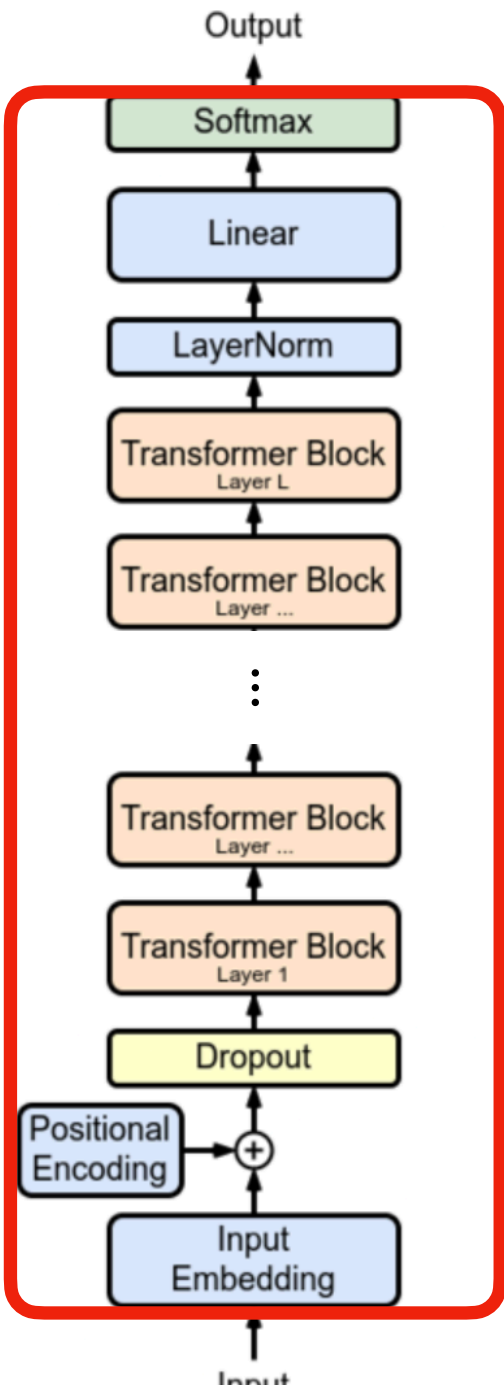
- ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ 1 (Correct)
- ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ 0 (Incorrect)
- ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ 1 (Correct)
- ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ 0 (Incorrect)



Policy Model

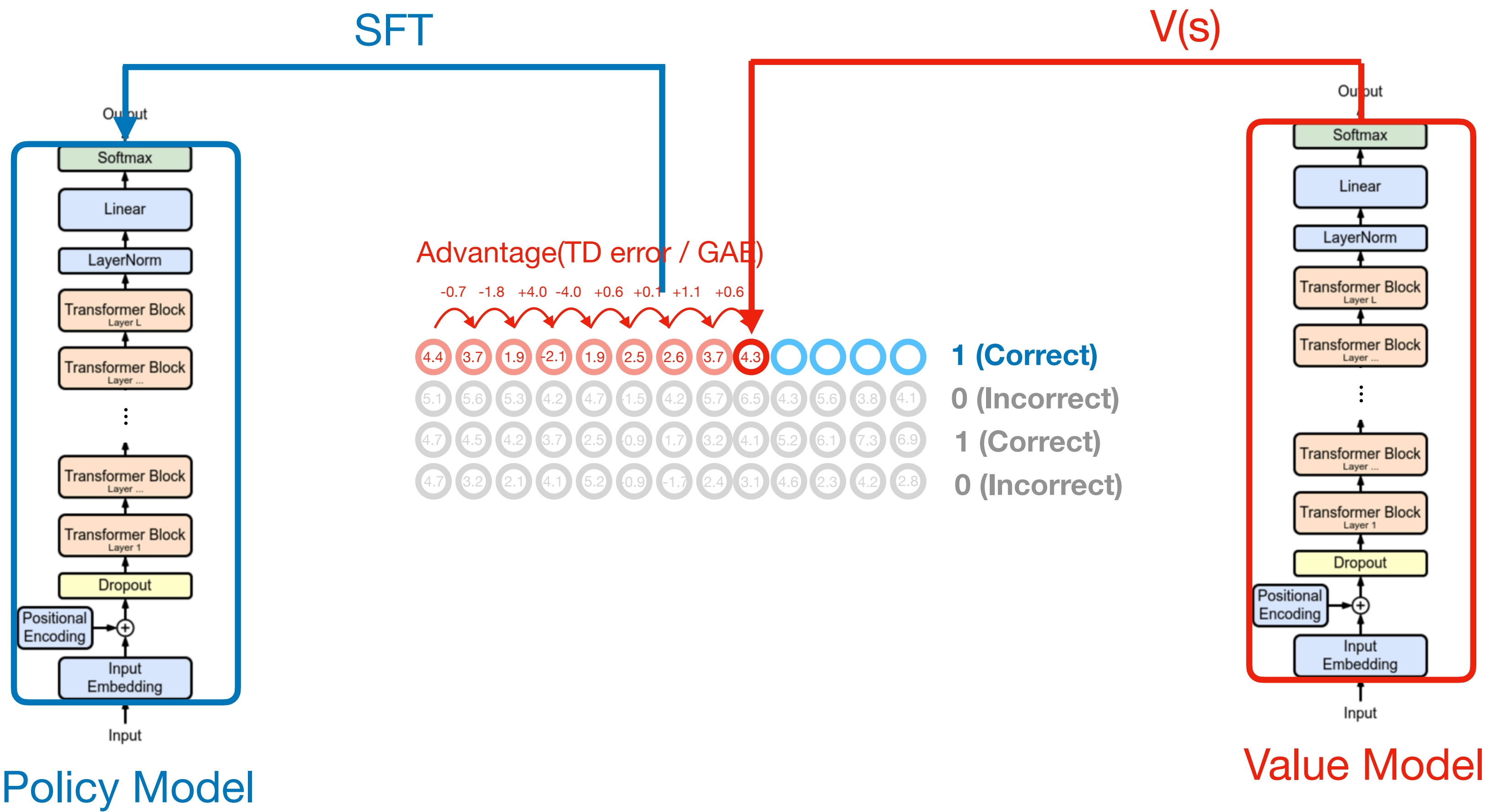


Reward Model



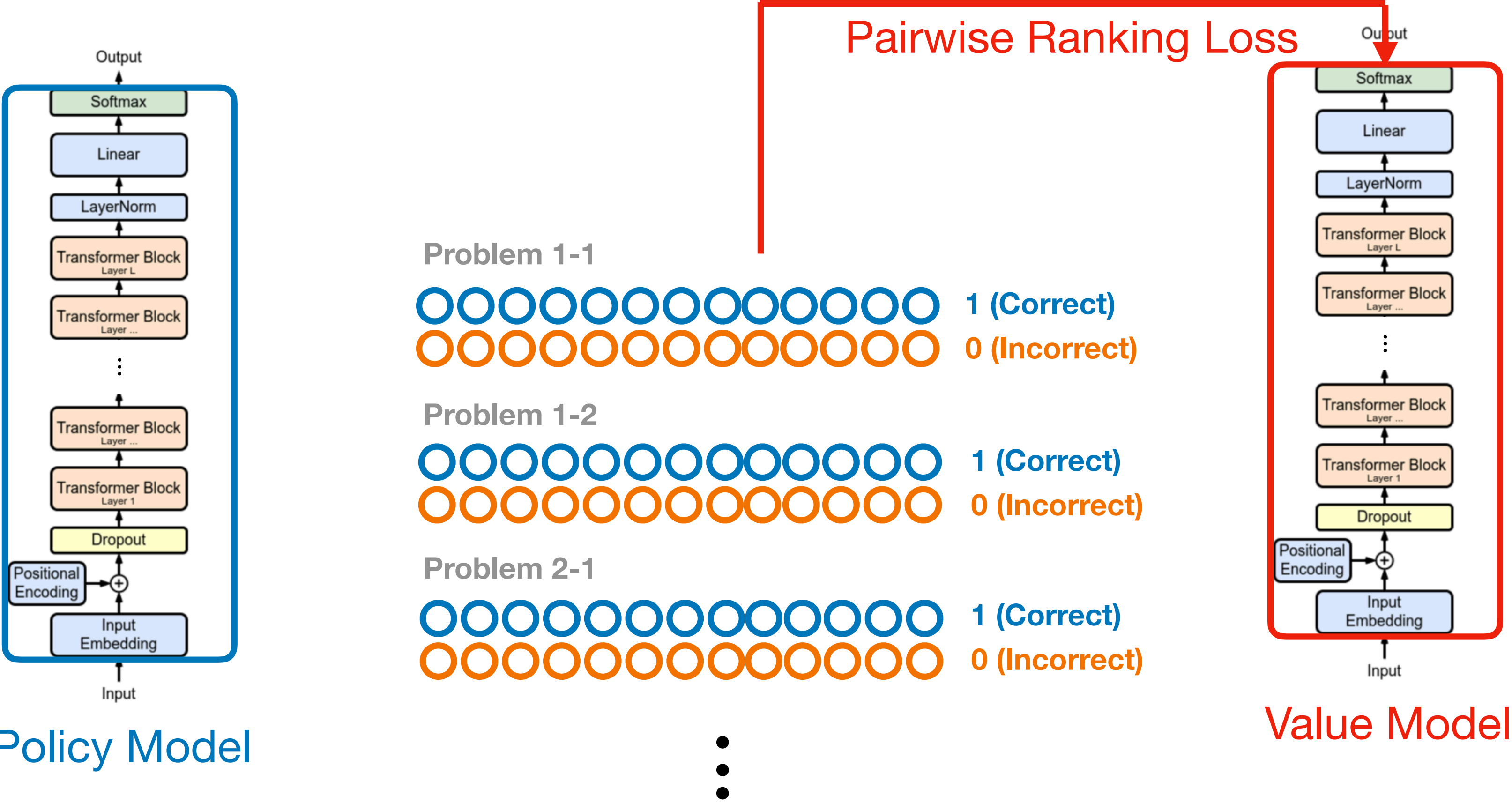
Value Model

SFT에 Value Model만 활용해보자 !



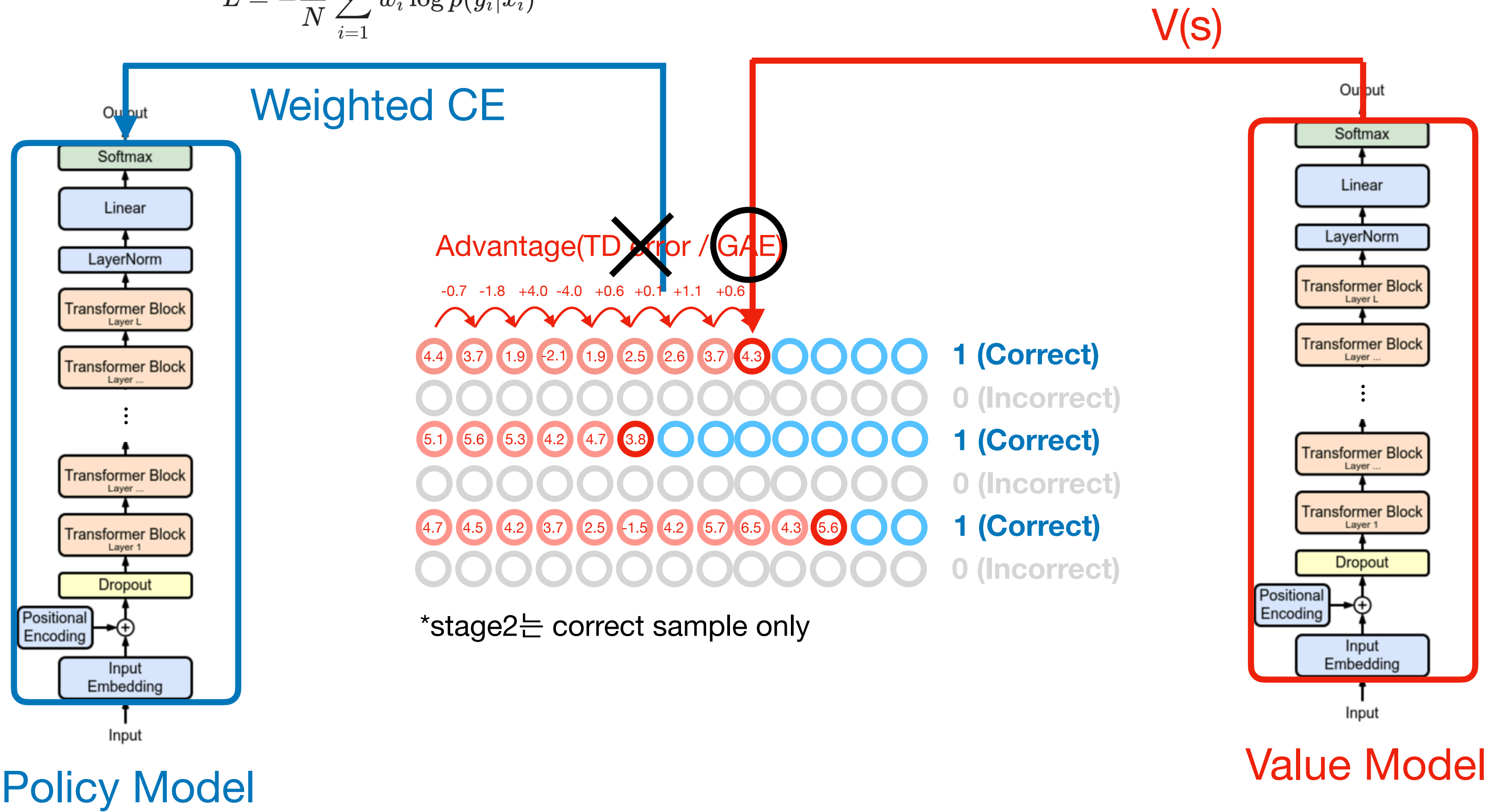
Stage 1. 정/오답 sample pair로 Value Model 학습

$$L(\phi) = -\log \sigma \left(\bar{V}_{\phi}(y_w) - \bar{V}_{\phi}(y_l) \right)$$



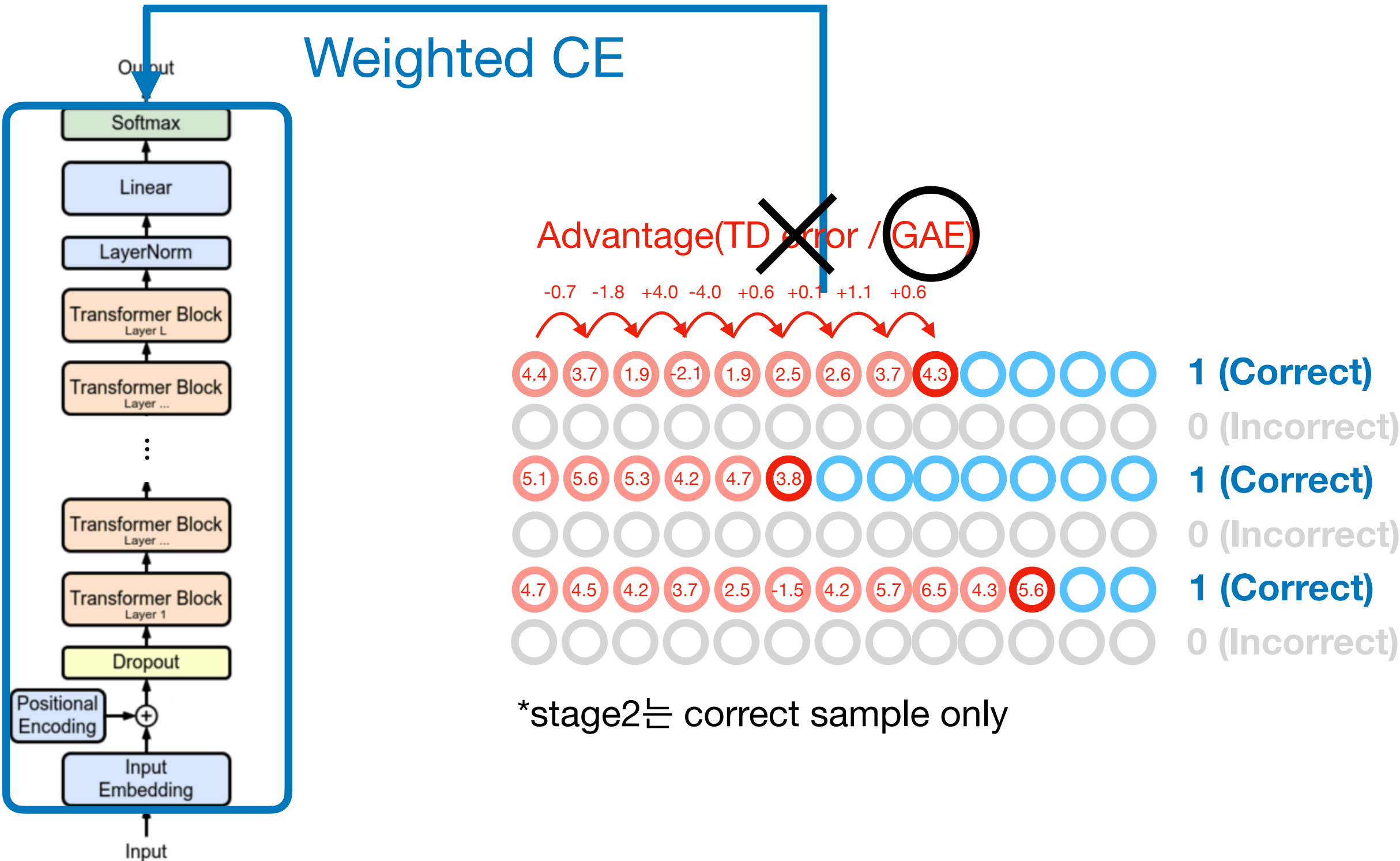
Stage 2. GAE 를 weight로 CE SFT

$$L = -\frac{1}{N} \sum_{i=1}^N w_i \log p(y_i|x_i)$$



Stage 2. GAE 를 weight로 CE SFT

$$L = -\frac{1}{N} \sum_{i=1}^N w_i \log p(y_i|x_i)$$



Policy Model

From GAE to Weight CE

*td error 대신 GAE 사용

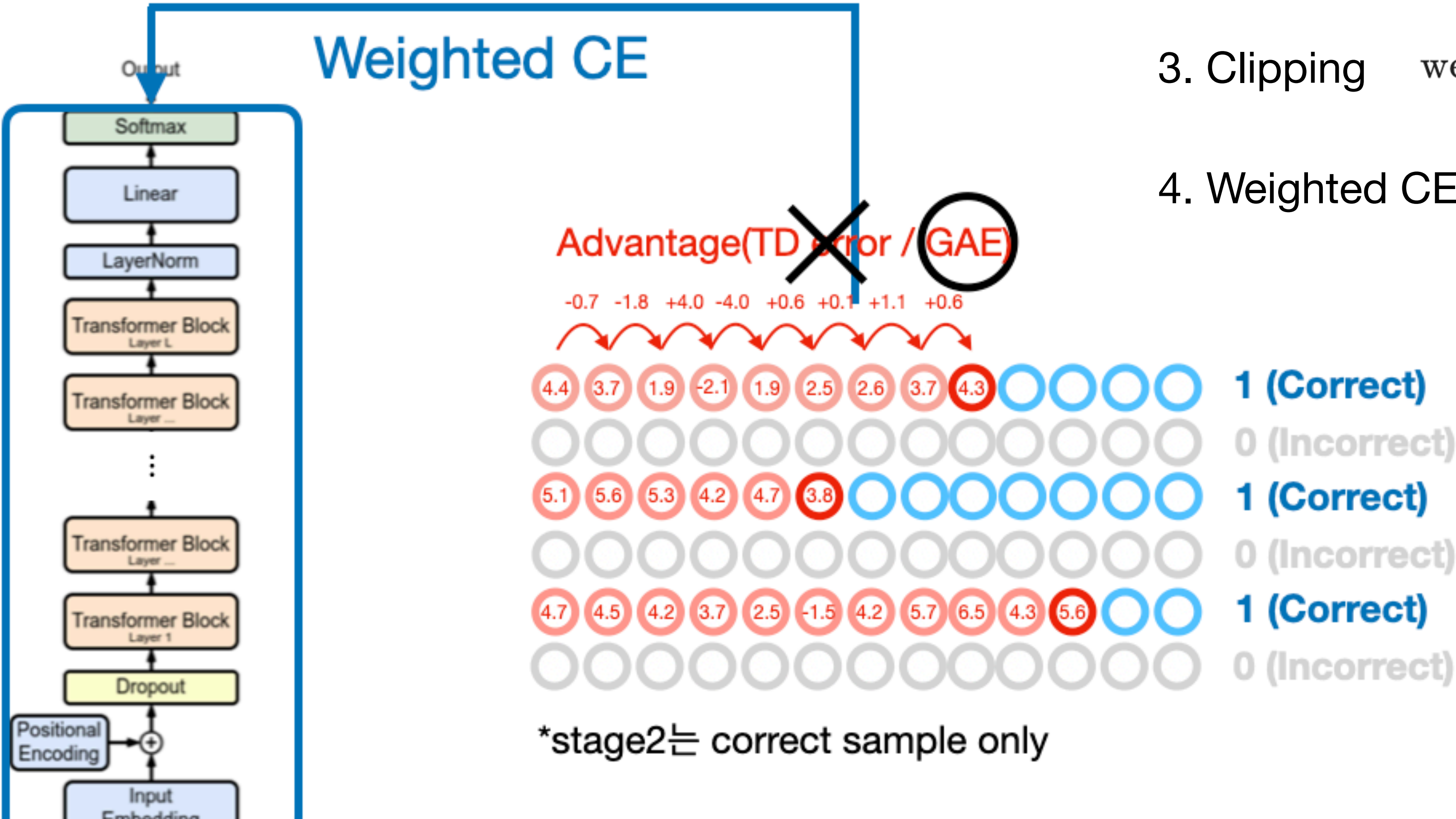
$$L = -\frac{1}{N} \sum_{i=1}^N w_i \log p(y_i|x_i)$$

1. Normalization (Batch Norm) $\hat{A}_i = \frac{A_i - \mathbb{E}[A]}{\sqrt{\text{Var}[A]}}$

2. Exponentiate $\exp(\hat{A}/\beta)$

3. Clipping $\text{weight} = \text{clip}(\exp(\hat{A}/\beta), 0.1, 3)$

4. Weighted CE $L = -\frac{1}{N} \sum_{i=1}^N w_i \log p(y_i|x_i)$



실무적 고려사항

1. Policy Model (MTP)과 Tokenizer가 완전히 동일한 Small Value Model이 필요했음

- 다행히 있었음. Llama2 기반 Sheard-Llama (2.7B)
- Pruning을 통해 모델 사이즈를 크게 줄이고 (7B >> 2.7B), 50B 토큰으로 대규모 SFT 훈련한 모델
- **Sheard-Llama** 모델에 **MLP layer**를 추가해 **Value Model**로 **Stage1** 학습

*3-layer MLP : 2560 >> 640 >> 320 >> 1 (1.84M)

2. H100 80GB 4GPU 환경, 최소한의 파라미터 & 데이터로 학습 진행

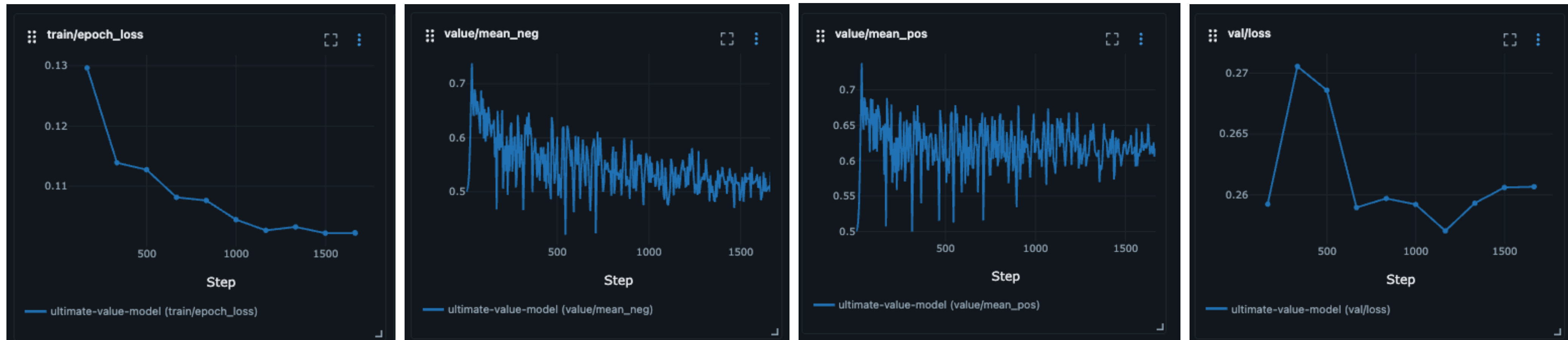
- Codecontests 데이터 총 360만 샘플, 전체 학습 현실적으로 불가 (논문에서는 전체 데이터 학습 진행)
- **20만 샘플만 1epoch** 학습 (전체 5% 가량)
- 7B 모델을 Full FineTuning 하기에 20만 샘플은 매우 적어 과적합이 발생함 >> LoRA 활용

*LoRA : Rank - 32, Alpha - 64, Target - q, k, v, o, gate, up, down(7개 모듈)

*policy model 160M, value model 51M parameters

Stage1. Value Model 학습

1. Train loss는 잘 떨어지는데, Validation loss가 거의 떨어지지 않는 문제 (과적합)



Run_id = c51e2b4ba4f84d1eaac8e862913220e1

데이터셋 문제임을 확인

1) CodeContests 데이터셋 360만 샘플 중, Unique Problem ID는 1만여개

>> Random Sampling 시 샘플 수가 많은 특정 Problem 샘플만 학습됨 >> 최대 동일 Problem Sample 60개로 제한

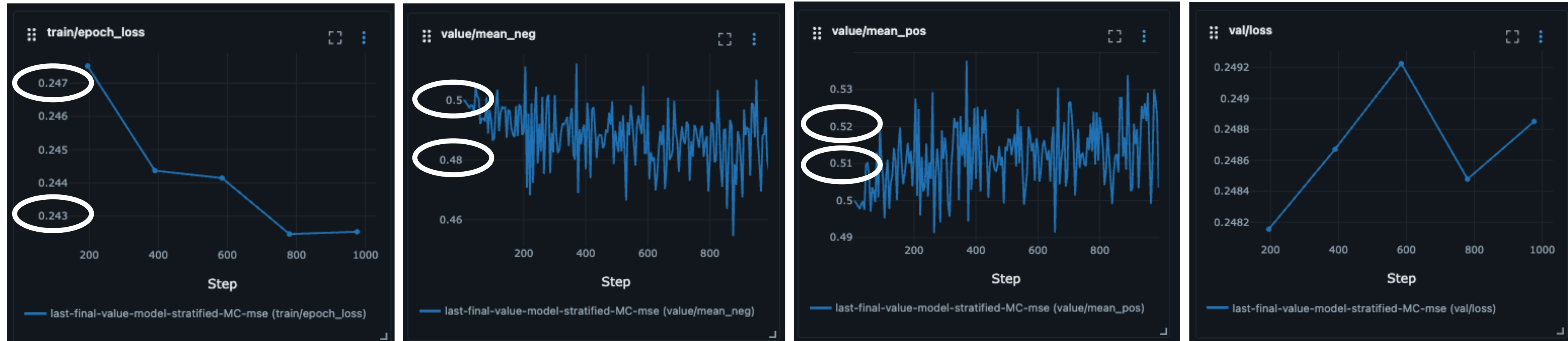
2) Correct - Incorrect 샘플의 평균 token수의 차이가 매우 큼 (170.7 vs 225.8)

>> Value model이 코드 논리를 배우는게 아니라, 학습한 problem id의 평균 토큰 길이를 학습하는 꼼수를 배우는 것으로 추정

>> 동일 problem ID에 대해, Difficulty X token size 로 binning해서 총화추출 진행

Stage1. Value Model 학습

2. 총화추출 진행 후, Train Loss가 거의 떨어지지 않음



Run_id = d9b057858dd74aad9628a65c09ece3f8

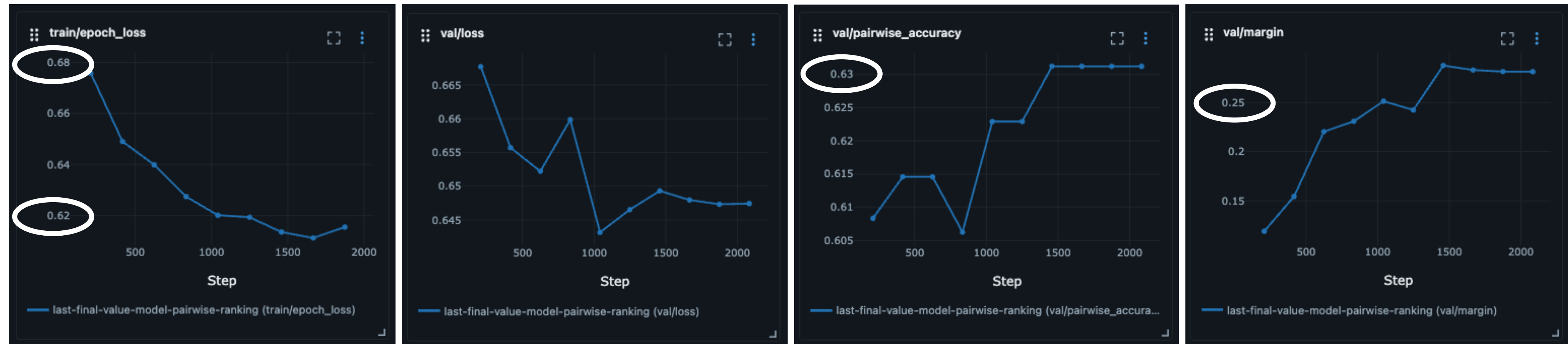
Code라는 도메인의 난이도가 매우 높음, MonteCarlo 방식의 Value Target으로 제대로 학습되지 않음

1) Lambda Return 방식 다양하게 활용해봤으나 여전히 실패

>> lambda 0.995, 0.997 등 다양한 수치 실험해봤으나 validation loss 및 accuracy가 유의미하게 변화하지 않음

Stage1. Value Model 학습

3. MSE 대신, Pairwise Ranking Loss 활용 >> 가장 유의미한 학습 결과



Run_id = 834aa63ec870415e9873b0b11c47963b

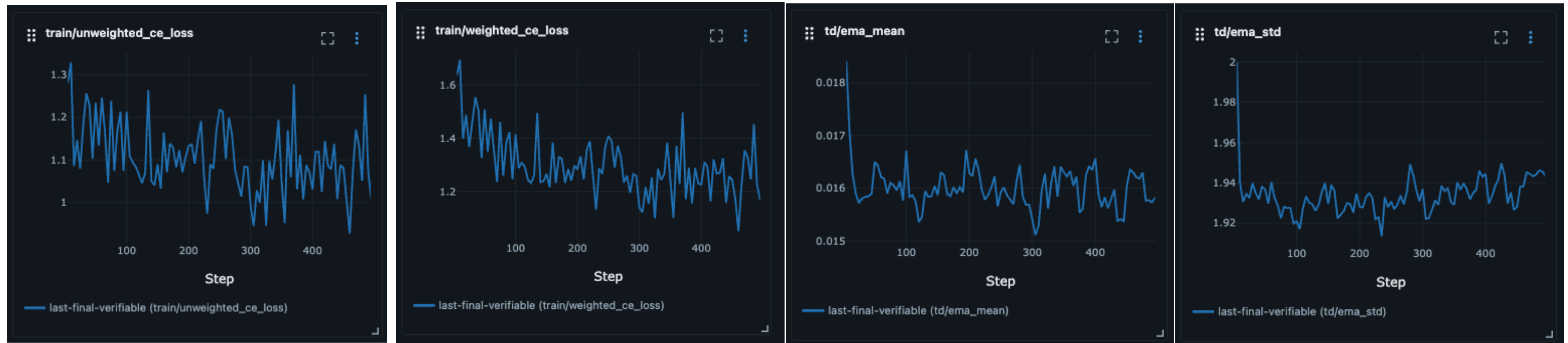
Code라는 도메인의 난이도가 매우 높음, MonteCarlo 방식의 Value Target으로 제대로 학습되지 않음

1) MSE 방식 보다 Pairwise Ranking Loss가 가장 유의미한 예측 결과를 보임

- validation accuracy > 0.63
- validation margin > 0.28

Stage2. MTP base model 학습

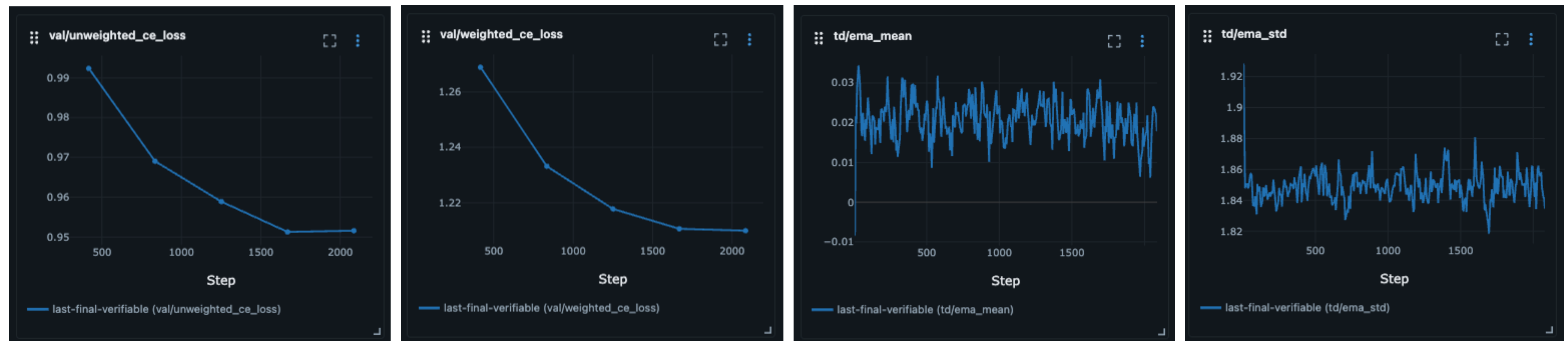
1. CE weighting 시, 단순 TD error 사용 >> train loss 패턴이 불안정 (학습 중단)



Run_id = 3c90453c783f44d4aa839de77fd70dbc

Stage2. MTP base model 학습

2. CE weighting 시, GAE를 사용하자 학습 안정화 성공



Run_id = b33455c660d743d5905b84a2ef278ba1

Result

Table 1: Pass@K Performance Comparison Across Training Methods

Dataset	Pass@1			Pass@5			Pass@10			Pass@20		
	MTP	SFT MTP	WCE MTP	MTP	SFT MTP	WCE MTP	MTP	SFT MTP	WCE MTP	MTP	SFT MTP	WCE MTP
HumanEval	2.94	2.56	3.25	13.17	11.85	14.27	23.30	21.45	24.79	37.50	35.00	40.00
MBPP	17.00	17.50	17.80	24.58	23.55	26.26	27.99	26.28	30.29	31.00	30.00	35.00
GSM8K	2.95	3.20	3.00	7.34	8.80	8.45	10.40	11.34	12.16	14.00	13.00	17.00
CodeContests	0.25	1.00	0.50	1.25	4.25	2.50	2.50	6.97	5.00	5.00	10.00	10.00
Average	5.79	6.06	6.14	11.59	12.11	12.87	16.05	16.51	18.06	21.88	22.00	25.50

MTP: Pre-training only | SFT MTP: Supervised fine-tuning | WCE MTP: Weighted Cross-Entropy training
Green highlighted cells indicate best performance per dataset-metric combination

*SFT & WCE 두 학습 시 Sample size, epoch, effective batch size, learning rate, activated parameter size 등 Configuration 모두 동일하게 맞춤

Result

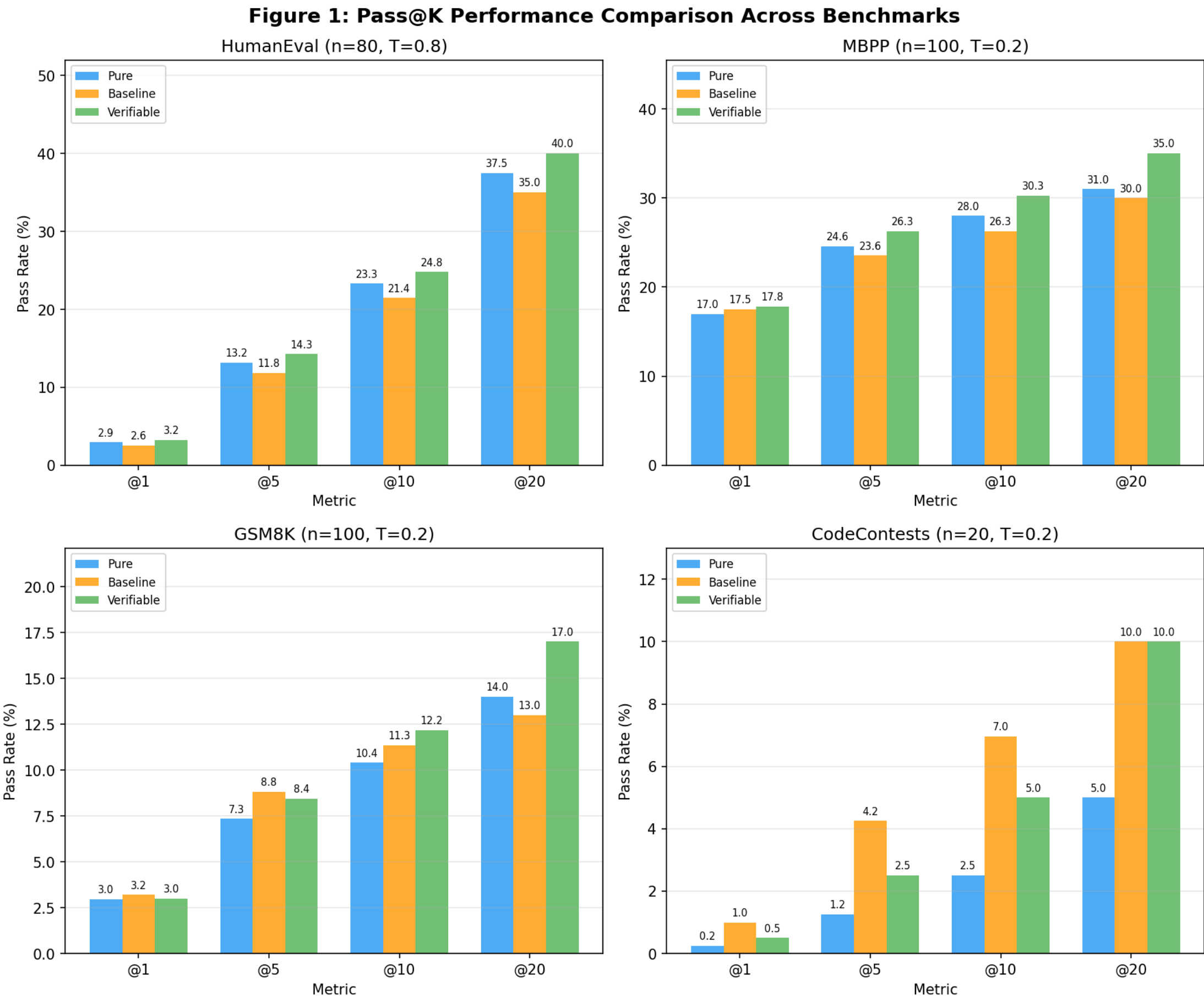
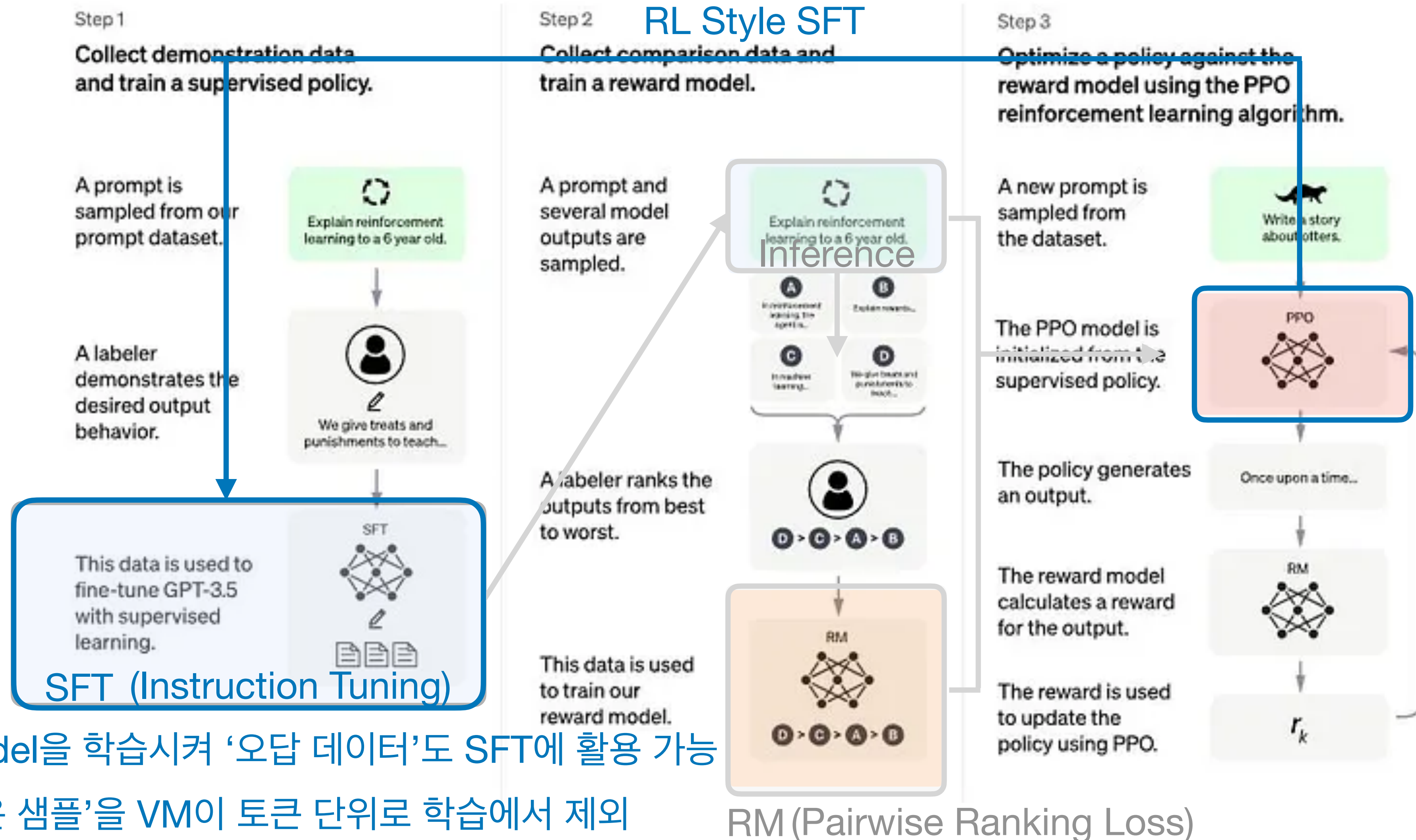


Table 2: Improvement of Verifiable (WMTF) over Baseline (SFT)

Dataset	Δ Pass@1	Δ Pass@5	Δ Pass@10	Δ Pass@20	Avg Δ
HumanEval	+0.69%	+2.42%	+3.34%	+5.00%	+2.86%
MBPP	+0.30%	+2.71%	+4.01%	+5.00%	+3.00%
GSM8K	-0.20%	-0.35%	+0.82%	+4.00%	+1.07%
CodeContests	-0.50%	-1.75%	-1.97%	0.00%	-1.05%

Green: Verifiable outperforms Baseline | Red: Baseline outperforms Verifiable

결국 우리는 무엇을 하고자 했던 것인가?



이번 연구의 한계 및 문제점

1. 왜 base 모델이 MTP여야 하는가?를 설명하지 못 함

- 이번 연구는 사실상 GAE를 활용한 weighted CE loss의 일반화 가능성을 보인 것이지, MTP여야 할 이유가 없었음
- Llama2 NTP 모델에도 GAE weighted CE가 유효한가? NTP에서보다 MTP에서 더 효과적인가? 확인 필요

2. 360만 Codecontests에서 20만 샘플(5%)만 활용

- 더 많은 샘플로 학습시 성능 차이가 더 커질 것인가? 줄어들 것인가
- Codecontests 도메인에서는 일반 SFT MTP의 성능이 더 높았음

3. Value Model 성능 및 효과 증명이 제한적

- Pairwise Ranking Loss로 Value Model accuracy 0.63 정도, Value Model의 고성능 달성에는 실패했음
- Stage 2의 성능이 실제로 Value Model의 토큰 가치 추정의 기여가 맞는가? 상세 확인이 필요함

이번 연구 및 프로젝트의 가치

1. 현대 RLHF 학습 과정을 일부 모사하여 새로운 SFT 학습 파이프라인을 완성

- Value Model 학습, td_error 및 GAE 계산 등
- LoRA, Batch Normalization 구현 등

2. 4 GPU 병렬학습에 필요한 다양한 기술 실현

- activation checkpointing, fsdp sharding 등 모델 병렬학습 파이프라인 구축 성공

3. 작은 샘플을 활용했으나, 가설 검증 가능성을 실제로 확인했음

- 학습한 샘플 사이즈가 CodeContests의 5%에 불과했으나, 가설에 부합하는 성과를 확인할 수 있었음

https://github.com/wooshikwon/weighted_mtp

전체 파이프라인 코드베이스, 27회 Mlflow 학습 기록 포함